

深圳大学实验报告

课程名称 计算机游戏开发

项目名称 实验 5 二维游戏原型设计

学 院 数学科学学院

专 业 信息与计算科学（数学与计算机实验班）

指导教师 储 颖

报 告 人 王曦 学号 2021192010

实验时间 2024 年 06 月 09 日

提交时间 2024 年 06 月 09 日

教务处制

目录

目录.....	2
一、实验目的与要求.....	3
二、实验内容与方法.....	3
三、实验步骤与过程.....	4
1. 构建项目.....	4
1.1 构建项目.....	4
1.2 摄像机调整.....	4
1.3 逻辑空物体.....	5
2. 创建苹果树、苹果、篮筐.....	7
2.1 模型获取.....	7
2.2 加载模型与创建预设体.....	7
2.3 设定物理图层与碰撞关系.....	15
3. 游戏的主逻辑.....	18
3.1 苹果树的逻辑.....	18
3.2 苹果的逻辑.....	22
3.3 篮筐的逻辑.....	22
4. 计分板.....	26
4.1 创建计分板.....	26
4.2 CurrentScore 的初始化与更新.....	28
4.3 HighScore 的初始化.....	30
5. Game Over.....	32
5.1 漏接苹果的逻辑.....	32
5.2 结束界面.....	34
6. 开始界面.....	41
6.1 新建场景.....	41
6.2 开始按钮点击事件.....	41
7. 修复游戏 Bug.....	44
8. 游戏优化.....	46
8.1 天空盒.....	46
8.2 BGM 与音效.....	46
8.3 键盘交互.....	48
8.4 颠倒重力方向.....	49
8.5 难度递增.....	50
四、实验结论或心得体会.....	52
9. 实验结论.....	52
10. 实验心得.....	52

一、实验目的与要求

1. 熟悉 Unity 引擎基本概念。
2. 掌握利用 Unity 引擎制作游戏原型的基本方法。
3. 理解 Unity 利用 C#语言进行编程的思想。

二、实验内容与方法

1. 完成游戏原型（40 分）

按照“游戏原型：《拾苹果》.pdf”步骤（P337-P366），实现完整的拾苹果游戏原型。

2. 完成修改内容一（5 分）

按实验文档 P367-后续工作-“欢迎界面”要求，添加游戏初始界面，要求包含游戏名称、“开始”按钮和个人学号姓名。点击“开始”按钮后，进入游戏。

3. 完成修改内容二（5 分）

按实验文档 P367-后续工作-“Game Over 界面”要求，添加游戏结束界面，要求包含“重新开始”按钮和最终得分展示，并提醒玩家是否打破了记录。

4. 游戏优化升级（5 分）

自行发挥想象力，优化和完善游戏功能。

5. 录制游戏视频（5 分）

录制游戏通关（失败）视频并上传。

6. 完成实验报告（40 分）

截图记录关键步骤，分析实验结果，撰写心得体会。

三、实验步骤与过程

（记录关键步骤/设计过程/设计结果的截图）

1. 构建项目

1.1 构建项目

在 Unity 中新建 3D 项目，在 Assets 文件夹中新建如图 1.1.1 所示的几个文件夹，其中：

- （1）Audio：存放音乐、音效资源。
- （2）Materials：存放材质。
- （3）Models：存放 3D 模型（含材质、纹理等）。
- （4）Prefabs：存放预制体。
- （5）Scenes：存放场景。
- （6）Scripts：存放脚本。

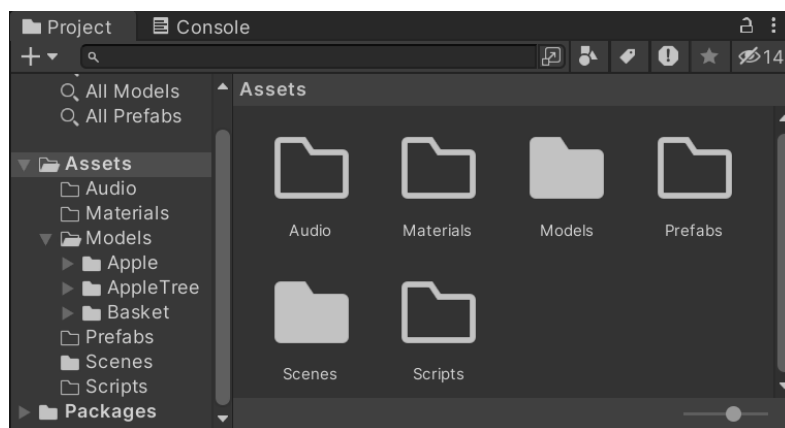


图 1.1.1：在 Assets 文件夹中新建文件夹

1.2 摄像机调整

《Apple Picker》基本是 2D 视角，故摄像机应采用正投影。

在 Hierarchy 窗口中选中“Main Camera”，在 Inspector 窗口中修改 Transform.Position 为 (0, 0, -10)，修改 Camera.Projection 为 Orthographic，Camera.Size 为 16。如图 1.2.1 所示。

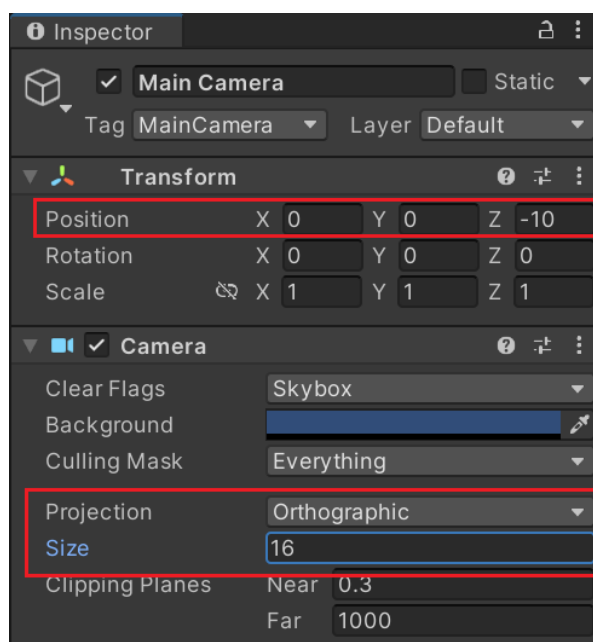


图 1.2.1: 修改 Main Camera 的位置、投影方式和大小

点击 Scene 窗口的菜单栏中的摄像机，设置相机广角 Field of View 为 30 ~ 40，以减小透视畸变。如图 1.2.2 所示。

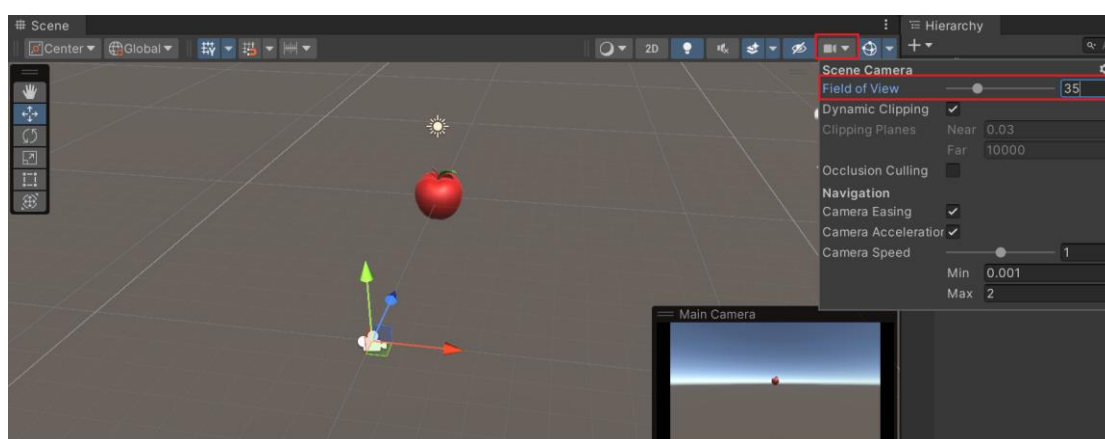


图 1.2.2: 调整 Main Camera 的广角

1.3 逻辑空物体

新建两个空物体 MainLogicObject 和 AudioLogicObject，分别用于后续挂载游戏主逻辑脚本和音频逻辑脚本。如图 1.3.1 所示。

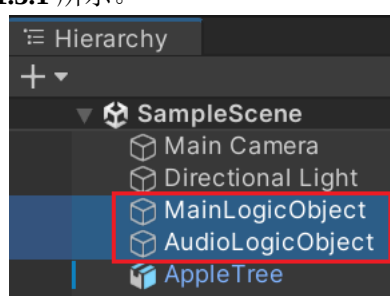


图 1.3.1: 新建逻辑空物体 MainLogicObject 和 AudioLogicObject
新建脚本 Logic_Main.cs，并挂在 MainLogicObject 上。

在 Logic_Main 类的 Start()函数中设置游戏的目标帧率 60，即约 16.7 ms 刷新一次。尽管实际运行时帧率无法稳定在 60，但 Unity 会尽量维持帧率在 60 附近。

```
void Start() {  
    Application.targetFrameRate = 60;  
}
```

2. 创建苹果树、苹果、篮筐

2.1 模型获取

在 Turbosquid 上下载如下三个免费模型：

(1) 苹果（如图 2.1.1 所示）：

<https://www.turbosquid.com/3d-models/apple-cartoon-3d-1495154>

(2) 苹果树（如图 2.1.2 所示）：

<https://www.turbosquid.com/3d-models/3d-model-tree-apple-1422666>

(3) 篮筐（如图 2.1.3 所示）：

<https://www.turbosquid.com/3d-models/3d-wooden-crate-model-1744087>



2.2 加载模型与创建预设体

下载上述模型的 FBX 格式，将它们拖入 Models 文件夹中。如图 2.2.1 所示。

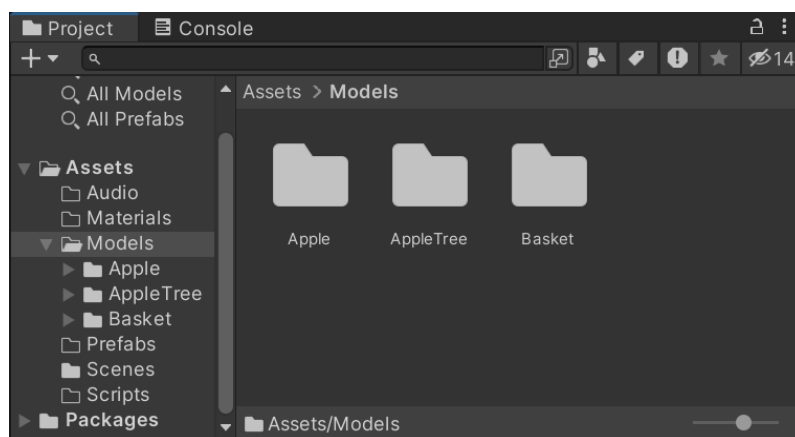


图 2.2.1: 将模型拖入 Models 文件夹

2.2.1 加载苹果树

以加载苹果树的模型为例。将 Models/AppleTree 文件夹中的 Apple_Tree.FBX 拖入 Hierarchy 面板，即可在 Scene 面板和 Game 面板中看到苹果树。如图 2.2.1.1 所示。

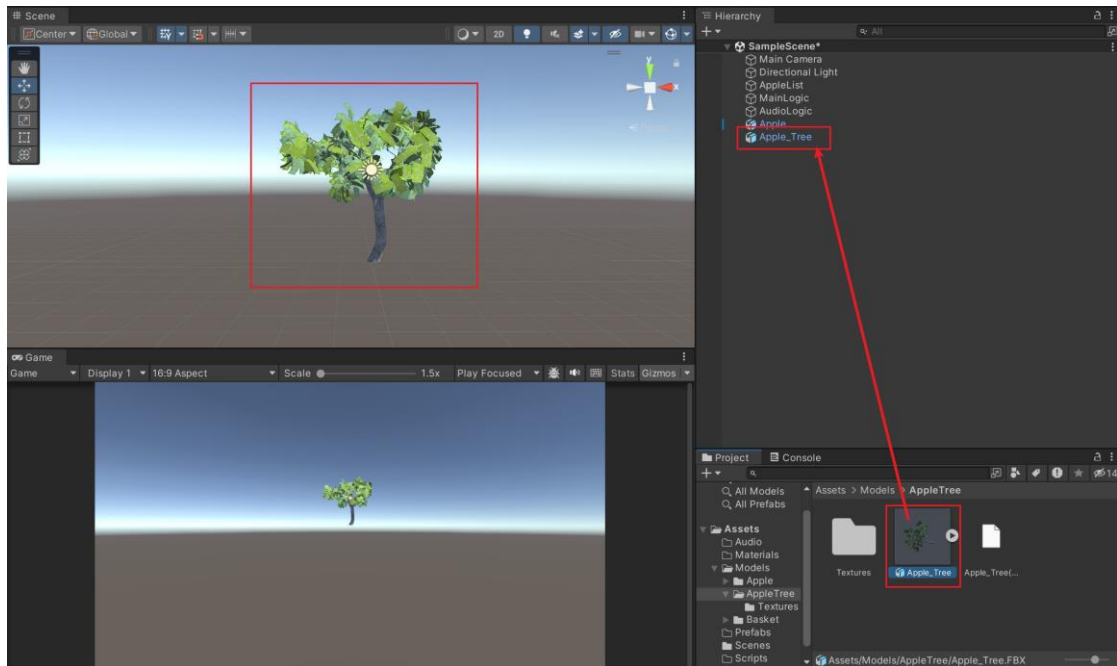


图 2.2.1.1：加载苹果树的模型

在 Hierarchy 面板中将其名称修改为“AppleTree”。在 Hierarchy 面板中选中 AppleTree，在 Inspector 面板中调节其 Position 和 Scale，把苹果树稍放大后置于（摄像机视角的）屏幕上方。如图 2.2.1.2 所示。

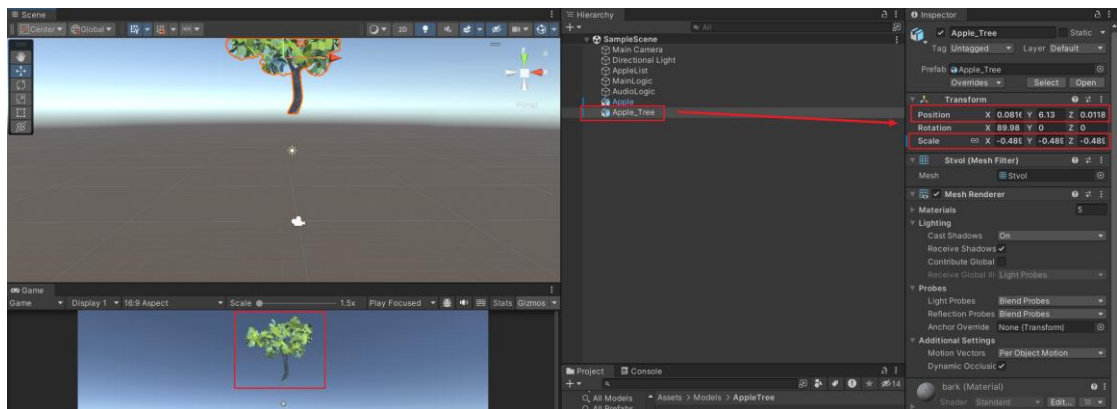


图 2.2.1.2：调整 AppleTree 的位置和大小

在 Inspector 面板中点击 Add Component，选择 Physics/Rigidbody，为 AppleTree 添加刚体组件。取消 Use Gravity 的勾选，否则 AppleTree 会被牛顿接管，而从天而降；勾选 Is Kinematic。如图 2.2.1.3 所示。

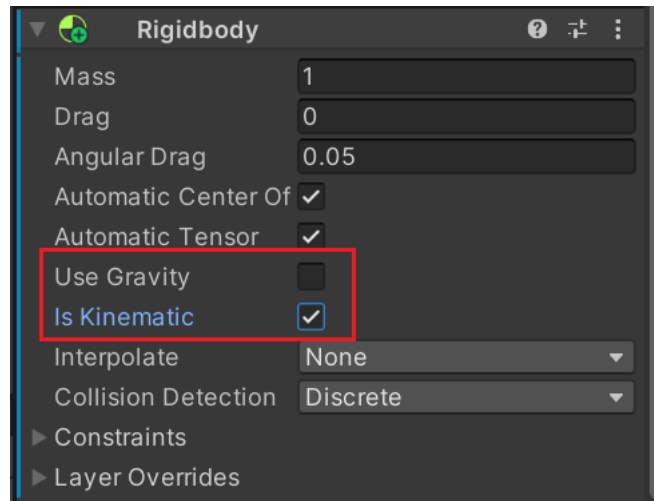


图 2.2.1.3: 取消 AppleTree 的重力

因为场景中只需要一棵苹果树，故没有必要将 AppleTree 生成预制体。

2.2.2 加载苹果

类似地将苹果的模型加入场景中。观察到苹果大小还算合适，无需调节。如图 2.2.2.1 所示。

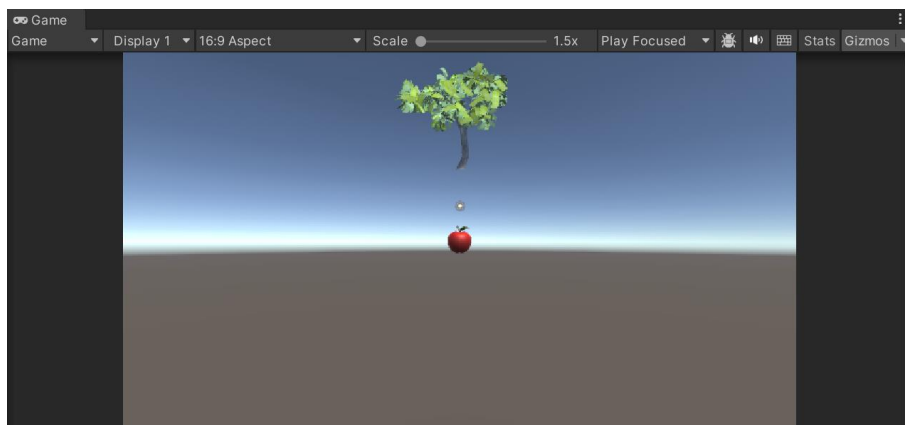


图 2.2.2.1: 加载苹果

类似地为 Apple 添加刚体组件，添加后点击运行按钮，在 Game 窗口中可观察到苹果作自由落体运动，一段时间后掉出屏幕外。如图 2.2.2.2 所示。

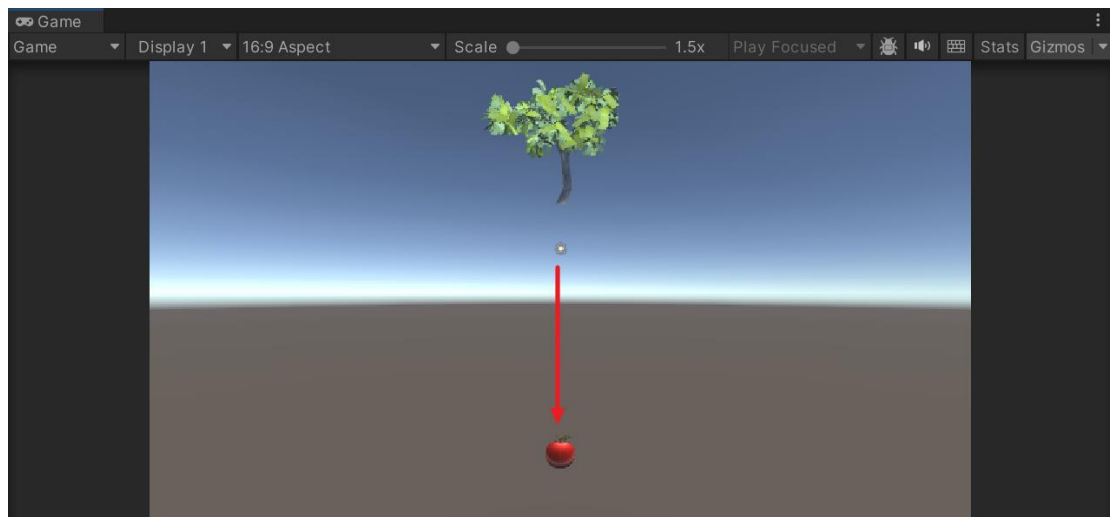


图 2.2.2.2: 苹果作自由落体运动

场景中可能会有多个苹果，后续将逐个苹果与篮筐作碰撞检测，故需获取当前场景中的所有苹果的数组（或列表、集合等）。此处有两种实现方式：

（1）新建一个空白对象 AppleList，后续将苹果的实例挂在 AppleList 上，则 AppleList 的所有儿子即为当前场景中的所有苹果。

（2）为 Apple 添加一个 Tag “Apple”，在脚本中获取所有 Tag 为 “Apple” 的游戏对象即为当前场景中的所有苹果。

下面采用方法（2）实现，尽管它的效率不如方法（1），因为方法（2）需要遍历场景中的所有游戏对象。

在 Hierarchy 面板中选中 Apple，在 Inspector 面板中点击 Tag/Add Tag...，如图 2.2.2.3 所示。

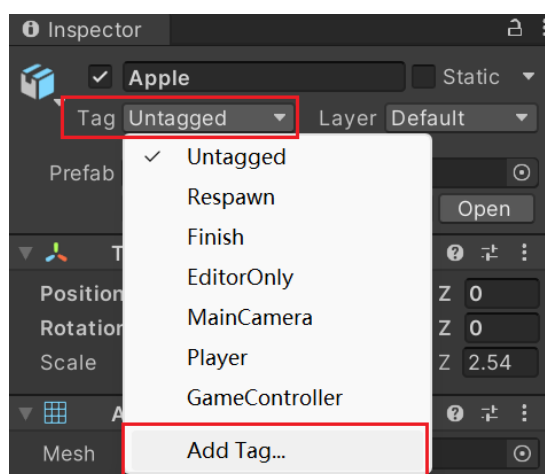


图 2.2.2.3: 在 Inspector 面板中点击 Tag/Add Tag...

点击加号，输入 Tag 名称 “Apple”，点击 Save。如图 2.2.2.4 所示。

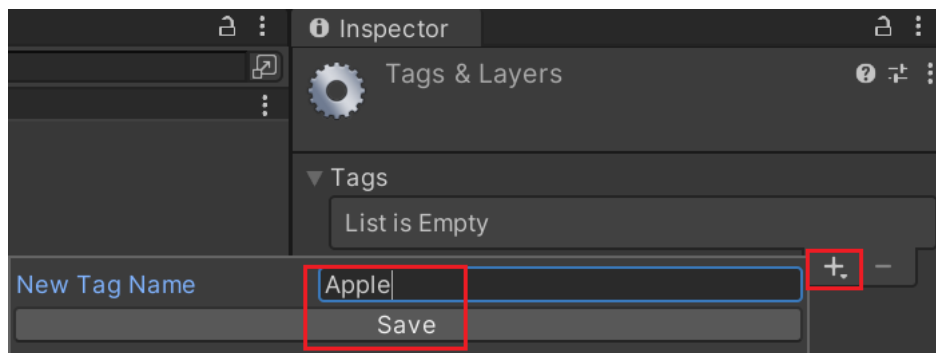


图 2.2.2.4: 新建 Tag “Apple”

回到 Apple 的 Inspector 面板，点击 Tag/Apple，设置 Apple 的 Tag 为 “Apple”。如图 2.2.2.5 所示。

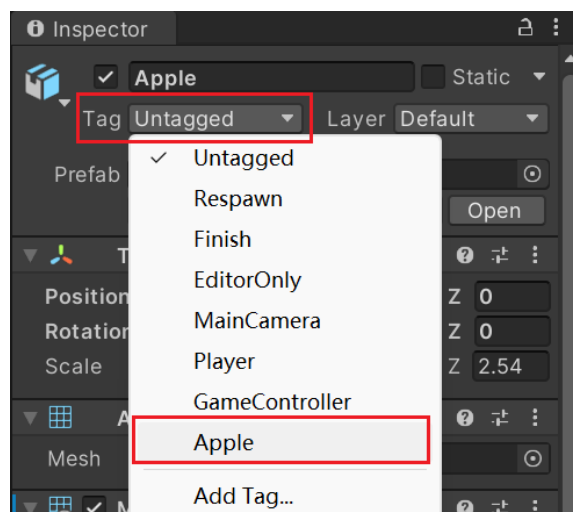


图 2.2.2.5: 设置 Apple 的 Tag 为 “Apple”

因 Apple 是自行导入的模型，不自带 Collider。在 Hierarchy 面板选中 Apple，在 Inspector 面板点击 Add Component/Physics/Sphere Collider，为 Apple 添加球形的 Collider。Collider 在 Scene 窗口和 Game 窗口中展示为绿色的手柄，拖动手柄可调节其大小，此处保持默认即可。如图 2.2.2.6 所示。

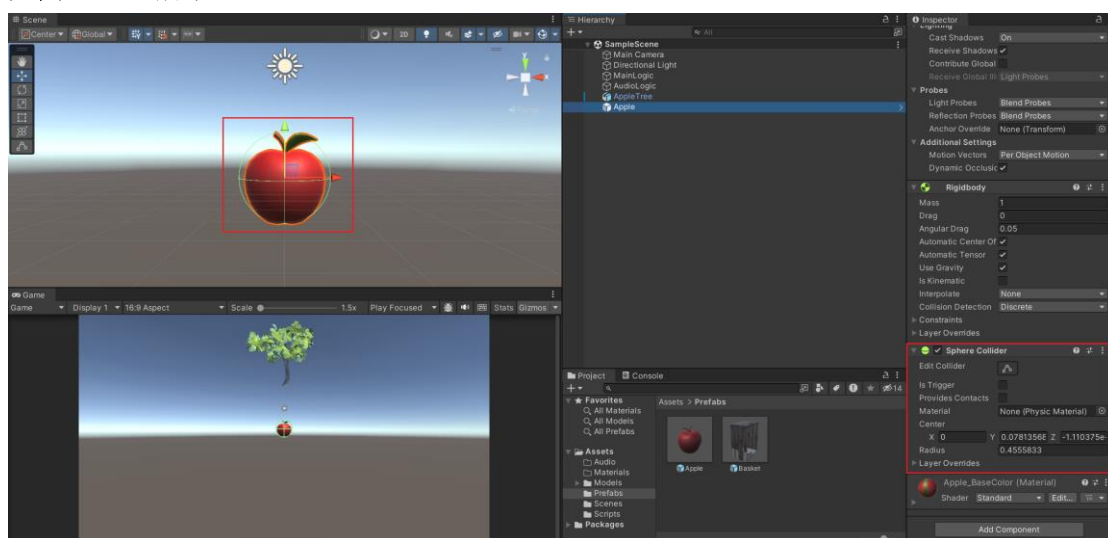


图 2.2.2.6: 为 Apple 添加球形的 Collider

将 Hierarchy 面板中的 Apple 拖到 Project 面板中的 Assets/Prefabs 文件夹中，创建 Apple 的预设体。如图 2.2.2.7 所示。随后即可删除场景中的 Apple。

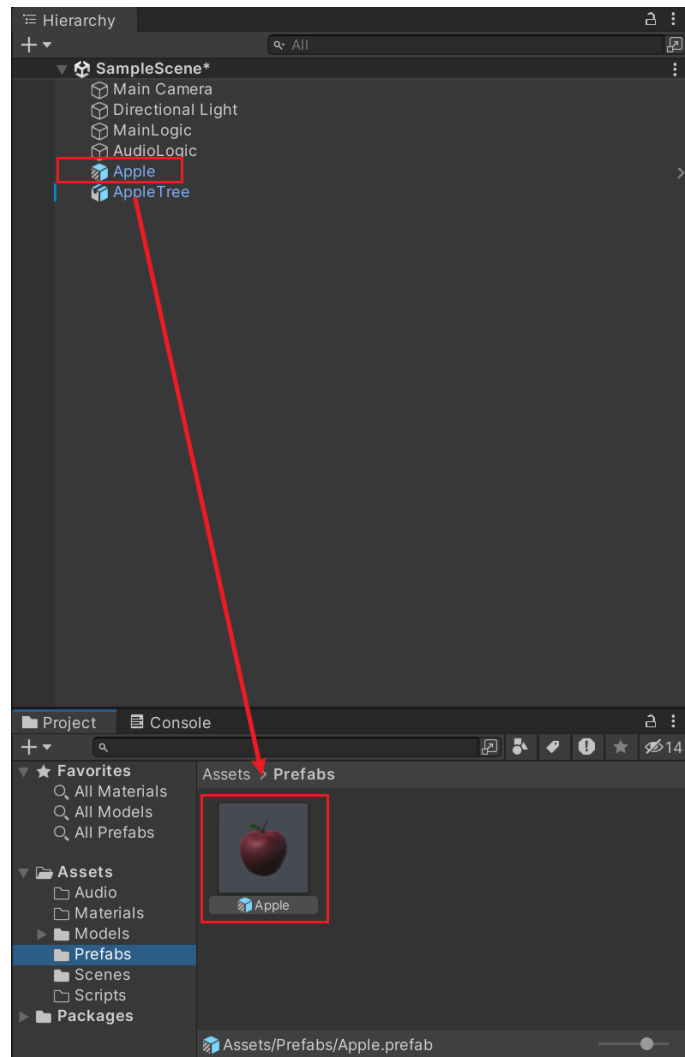


图 2.2.2.7: 创建 Apple 的预设体

2.2.3 加载篮筐

类似地加载篮筐的模型，观察到它没有自动绑定材质，且篮筐的角度不正确。如图 2.2.3.1 所示。

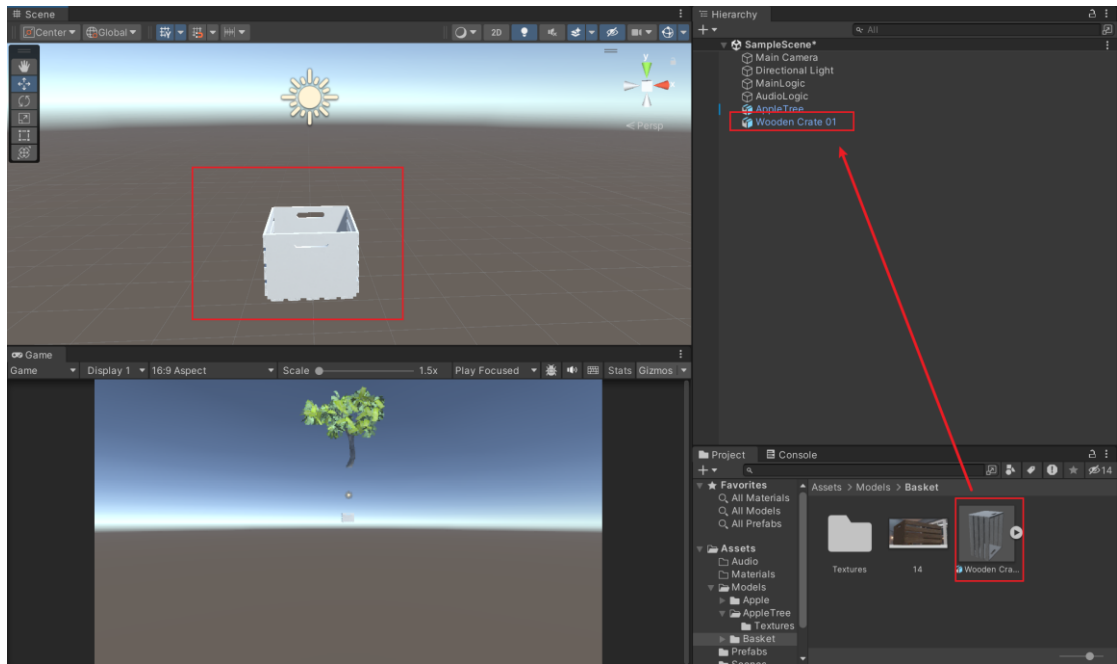


图 2.2.3.1：加载篮筐的模型

在 Hierarchy 面板中将“Wooden_Crate 01”重命名为“Basket”。将 Project 窗口中的 Basket/Materials 中的 14.mat 拖到 Hierarchy 面板中的 Basket 上。在 Inspector 面板中修改 Transform.Rotation 和 Transform.Scale，将篮筐调整为合适的角度和大小。如图 2.2.3.2 所示。

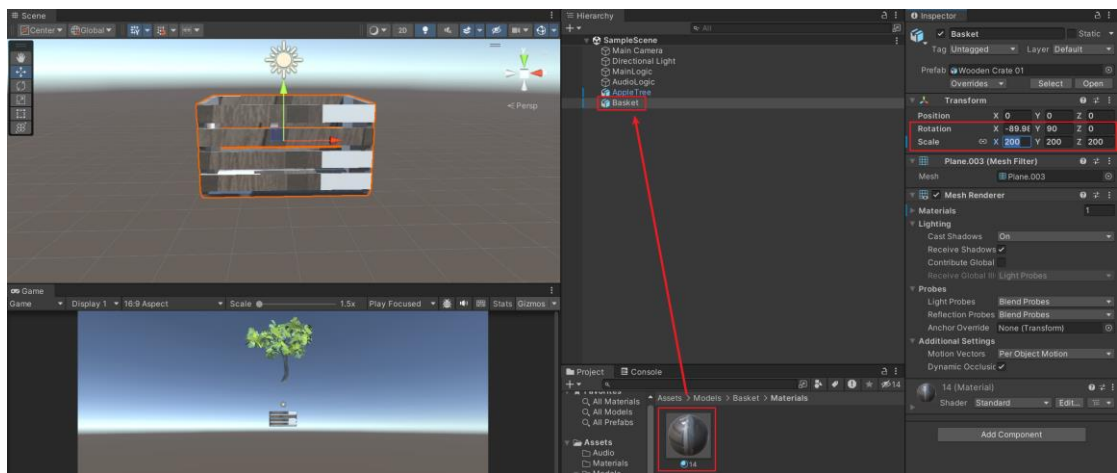


图 2.2.3.2：为篮筐添加材质，并调整角度和大小

类似地为 Basket 添加刚体组件，关闭 Use Gravity，启用 Is Kinematic。如图 2.2.3.3 所示。

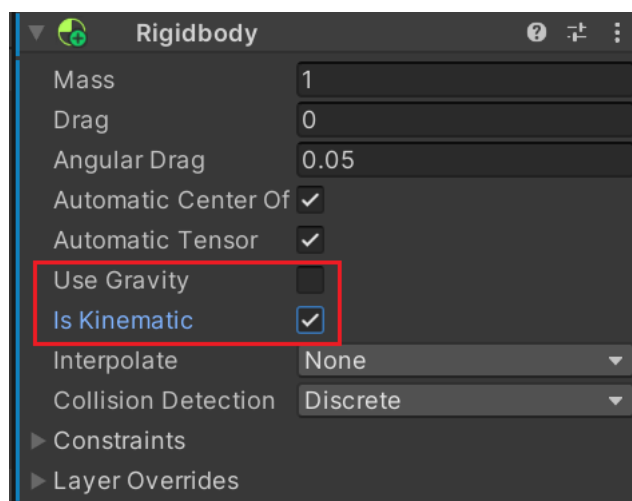
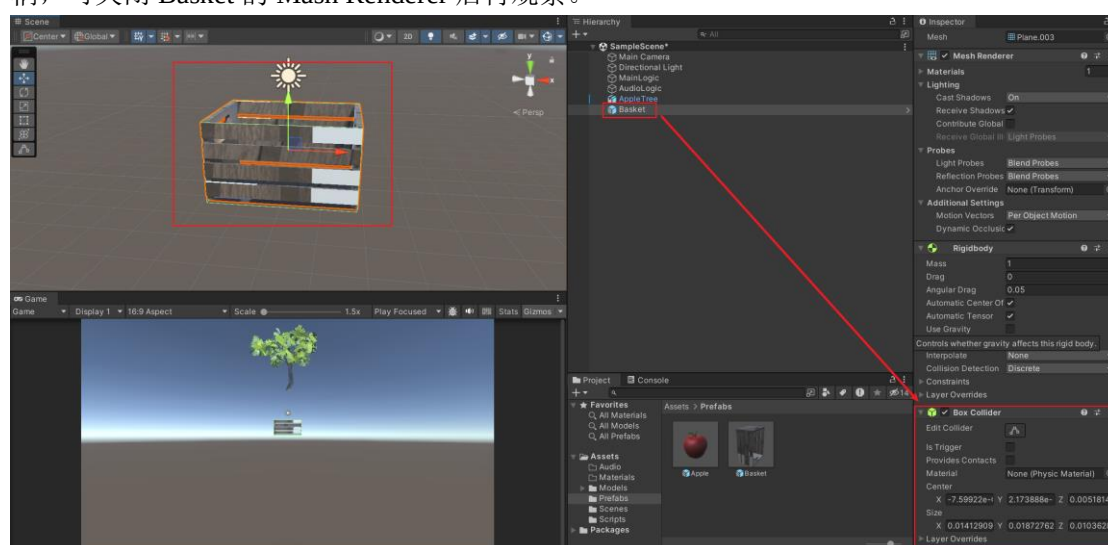


图 2.2.3.3: 关闭 Use Gravity, 启用 Is Kinematic

同理为篮筐添加 Box Collider 并调整大小。如图 2.2.3.4 所示。若调整时看不清绿色的手柄, 可关闭 Basket 的 Mesh Renderer 后再观察。



将 Hierarchy 窗口中的 Basket 拖到 Project 窗口中的 Assets/Prefabs 文件夹中, 创建 Basket 的预制体。如图 2.2.3.5 所示。随后删除场景中的 Basket。

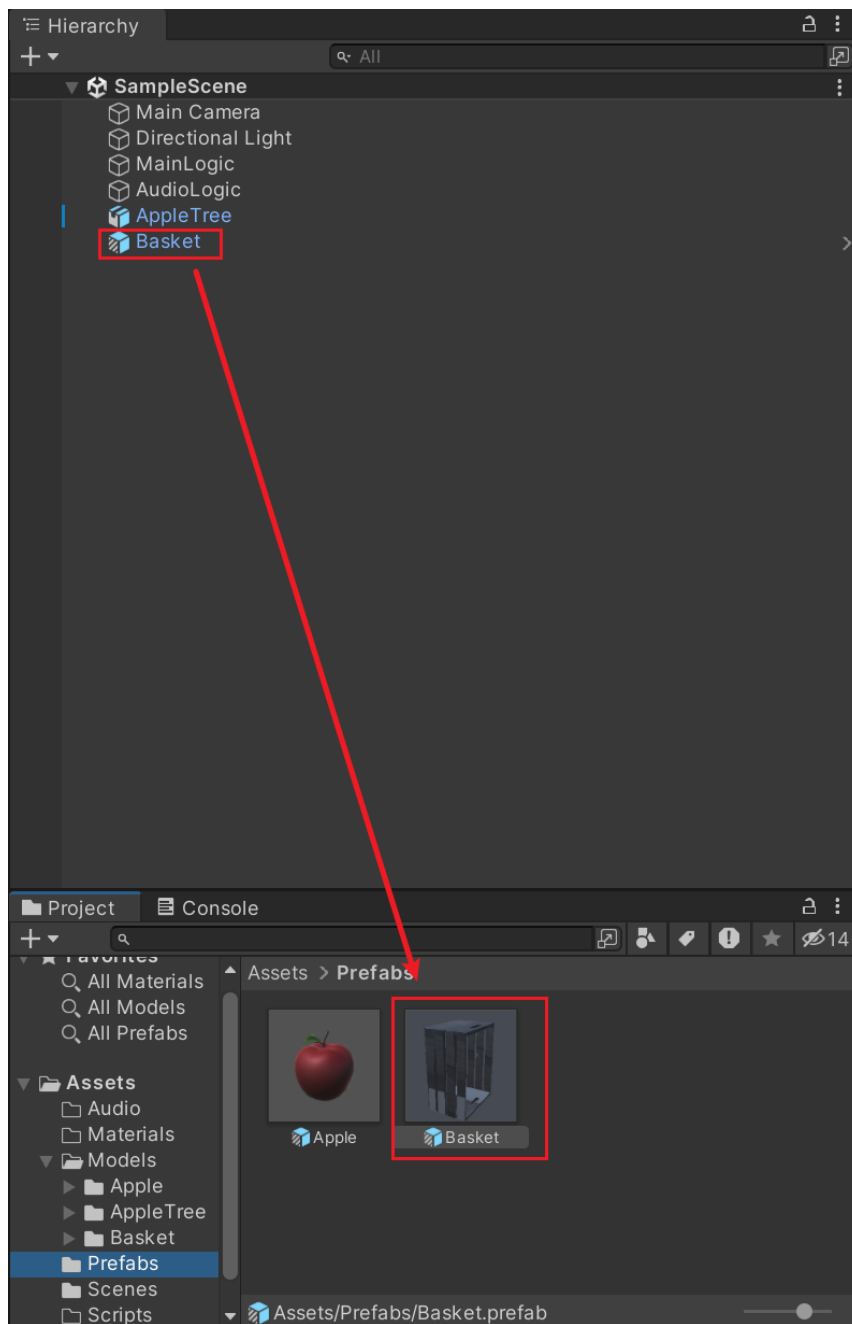


图 2.2.3.5: 创建 Basket 的预制体

2.3 设定物理图层与碰撞关系

《Apple Picker》只需要苹果与篮筐的碰撞，不需要苹果与苹果树、苹果与苹果的碰撞。为此，可将它们设置在不同的物理图层，并调整碰撞关系。

在 Hierarchy 面板中选中 AppleTree，在 Inspector 面板中点击 Layer/Add Layer...。如图 2.3.1 所示。

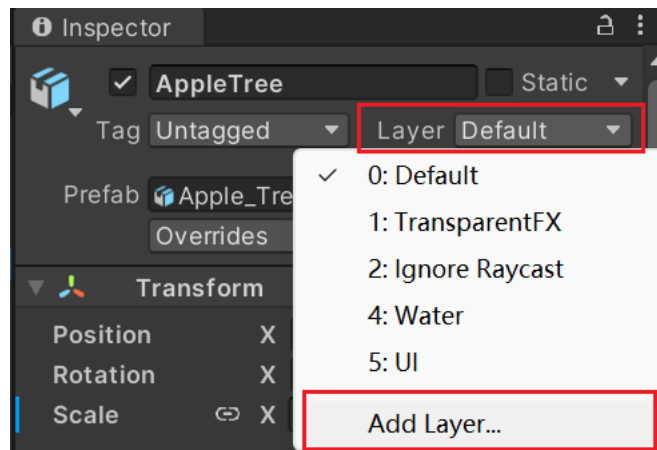


图 2.3.1: 在 Inspector 面板中点击 Layer/Add Layer...

设置 User Layer 8 为 AppleTree, User Layer 9 为 Apple, User Layer 10 为 Basket。如图 2.3.2 所示。



图 2.3.2: 添加三个 User Layer

点击左上角菜单栏中的 Edit, 点击 Project Settings/Physics, 取消 Apple 与 AppleTree 和 Apple 的碰撞。如图 2.3.3 所示。

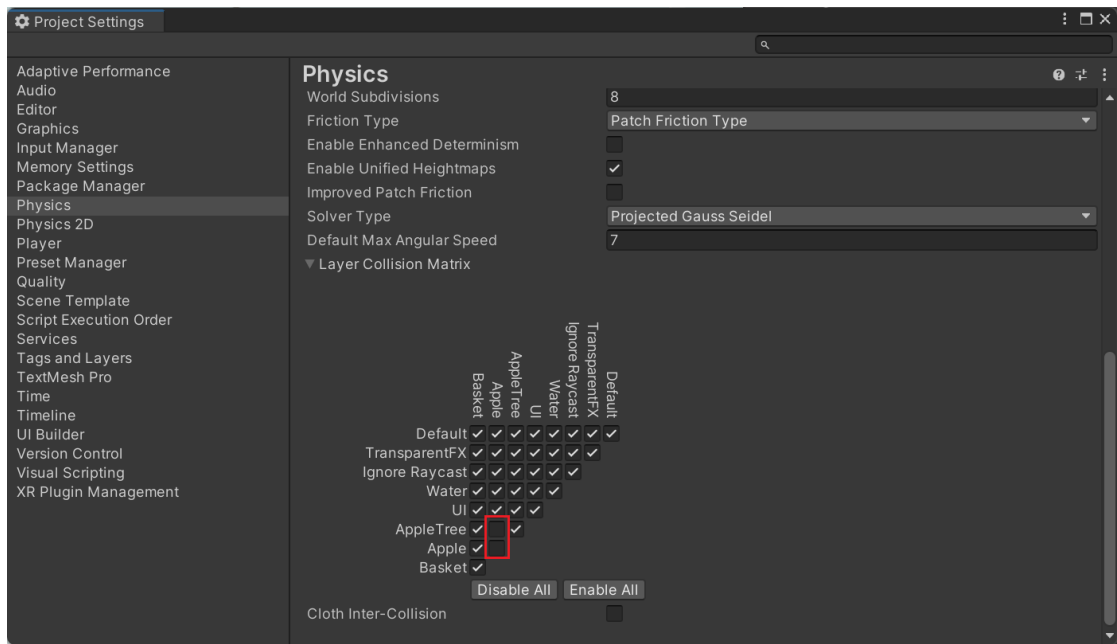


图 2.3.3: 取消 Apple 与 AppleTree 和 Apple 的碰撞

3. 游戏的主逻辑

3.1 苹果树的逻辑

3.1.1 概述

苹果树包含如下的逻辑：

- (1) 在 x 方向的一定范围内左右移动。
- (2) 每隔一段时间有一定概率转向。
- (3) 每隔一段时间掉落一个苹果。

在 Project 面板中的 Assets/Scripts 文件夹中新建脚本 Logic_AppleTree。如图 3.1.1.1 所示。

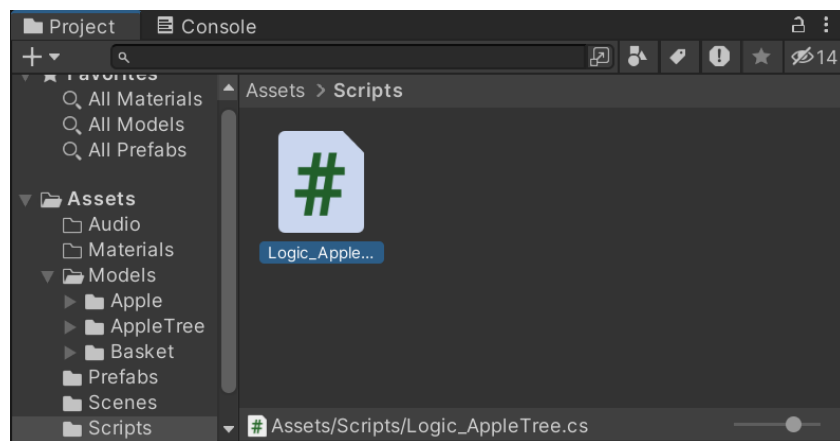


图 3.1.1.1: 新建脚本 Logic_AppleTree

将 Logic_AppleTree.cs 拖到 AppleTree 上，即将该脚本挂到苹果树上。如图 3.1.1.2 所示。

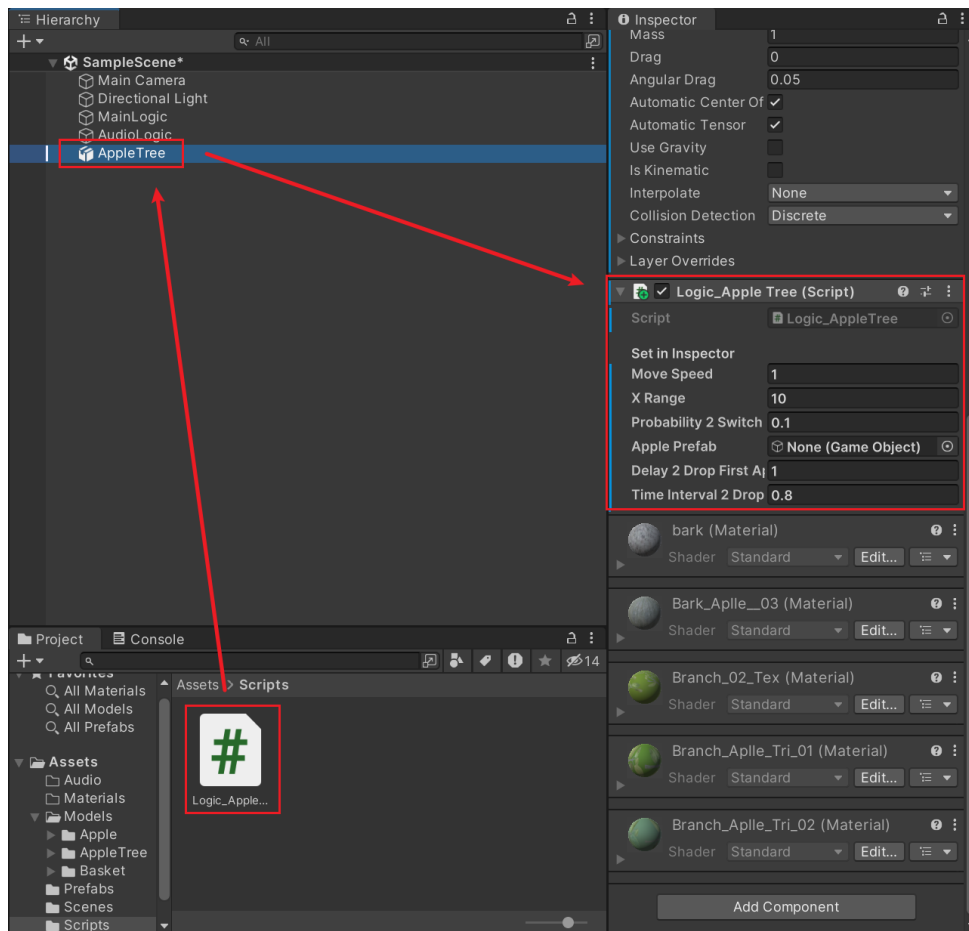


图 3.1.1.2: 将脚本 Logic_AppleTree 挂到 AppleTree 上

双击脚本即可用 Visual Studio 打开。初次打开可能需要安装 Unity 组件，跟随提示安装后，重新生成项目即可。

3.1.2 成员变量

在 Logic_AppleTree 类中加入如下成员变量。其中 public 表示这些属性可在 Inspector 面板中设置，[Header("Set in Inspector")] 是这些属性在 Inspector 面板中的头部，如图 3.1.2.1 所示。

```
[Header("Set in Inspector")]
```

```
// 苹果树的移动
```

```
public float moveSpeed = 10.0f; // 苹果树移动速度 (单位 / 秒)
```

```
public float xRange = 20.0f; // 苹果树移动范围为 ±leftAndRightEdge
```

```
public float probability2SwitchDirection = 0.02f; // 苹果树转向的概率
```

```
// 苹果树掉落苹果
```

```
public GameObject applePrefab; // 苹果预设体
```

```
public float delay2DropFirstApple = 1.0f; // 掉落第一个苹果的时间间隔
```

```
public float timeInterval2DropApples = 0.8f; // 除第一个苹果外，掉落苹果的时间间隔
```



图 3.1.2.1: 属性在 Inspector 面板中的头部

3.1.3 掉落苹果

在 Start()函数中用 InvokeRepeating()实现第一次隔时间 delay2DropFirstApple 调用函数 DropApple(),随后每隔 timeInterval2DropApples 时间调用函数 DropApple(),其中 DropApple()函数实现根据苹果的预设体生成一个实例,并将其位置设置为苹果树的中心点的位置。

若要求苹果在苹果树的根部掉落,可在 Hierarchy 面板中 AppleTree 对象下新建一个空的子对象,调整其位置为 AppleTree 的根部,并以其位置为苹果的出生点。此处略。

```
void Start() {
    InvokeRepeating("DropApple", delay2DropFirstApple, timeInterval2DropApples);
}

void DropApple() {
    GameObject apple = Instantiate<GameObject>(applePrefab);
    apple.transform.position = transform.position;
}
```

在 Update()函数中更新 AppleTree 的 x 坐标,超出边界时转向。为保证每隔一段时间转向的逻辑与运行速度无关,将其放在 FixedUpdate()函数中。

```
void Update() {
    transform.Translate(moveSpeed * Time.deltaTime, 0.0f, 0.0f);

    // 超过边界转向
    float x = transform.position.x;
    if (x < -xRange || x > xRange) {
        moveSpeed = -moveSpeed;
    }
}

void FixedUpdate() {
    // 10% 的概率转向
    if (Random.value < probability2SwitchDirection) {
        moveSpeed *= -1;
    }
}
```

在 Hierarchy 面板中选择 AppleTree, 将 Project 面板的 Assets/Prefabs 文件夹中的 Apple

（左键按住不松手）拖到 Inspector 面板的 Logic_Apple Tree (Script)/Apple Prefab 处，设置苹果的预设体。如图 3.1.3.1 所示。

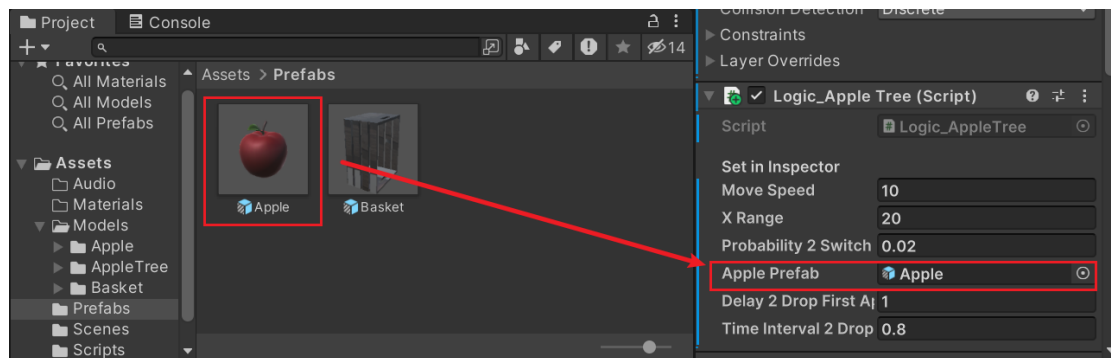


图 3.1.3.1: 设置苹果的预设体

运行，在 Game 面板中观察到苹果树左右移动，并掉落苹果。在 Hierarchy 面板中可看到生成的苹果实例。如图 3.1.3.2 所示。

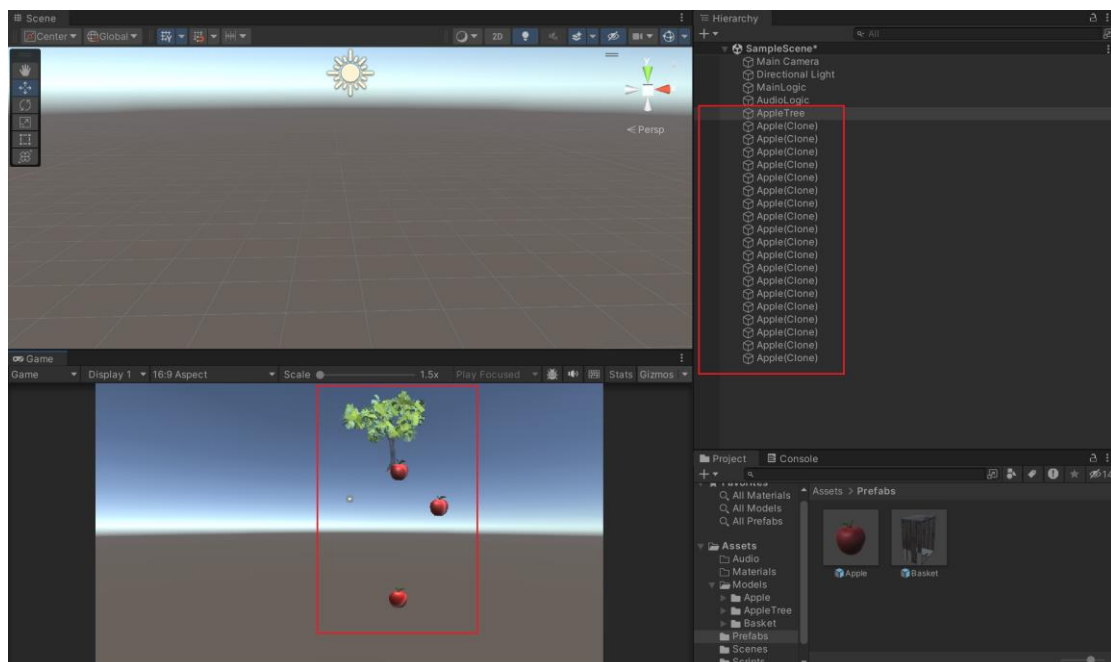


图 3.1.3.2: 苹果树的逻辑运行情况

在场景中加入一个 Basket 并调整到合适的位置，观察到苹果与篮筐可发生碰撞，证明 Collider 的设置无误。如图 3.1.3.3 所示。

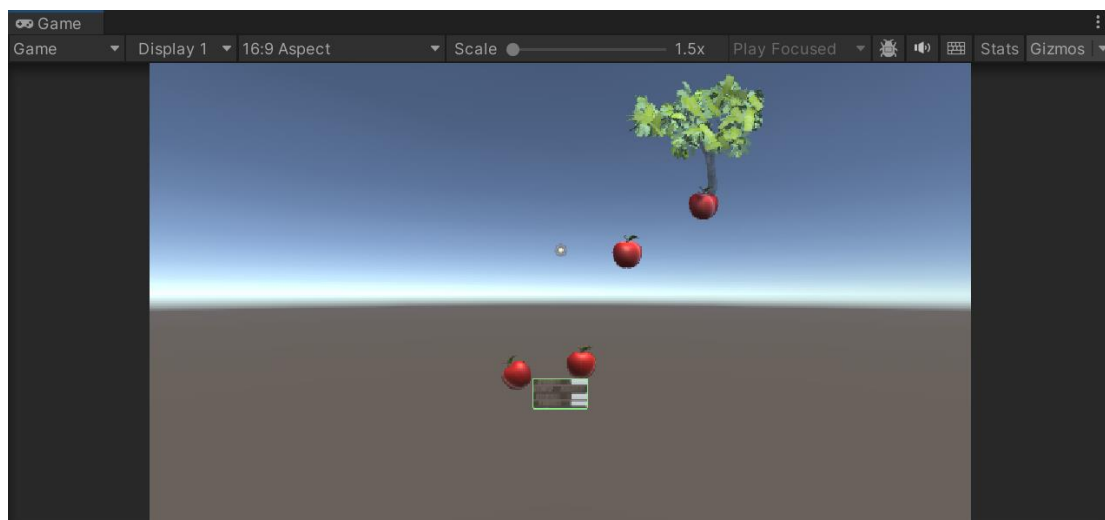


图 3.1.3.3: 苹果与篮筐可发生碰撞

3.2 苹果的逻辑

苹果的逻辑为：掉落一定距离后自毁。

新建脚本 `Logic_Apple.cs`，并挂在 `Apple` 的预制体上。

`Logic_Apple` 类中的成员变量 `yMin` 设置最小 `y` 值，苹果掉落至 `y` 小于该值后自毁。注意需销毁的是 `this.gameObject`，因为 `this` 是该脚本。

```
public class Logic_Apple : MonoBehaviour {
    [Header("Set in Inspector")]
    public float yMin = -25.0f; // 最小 y 值

    void Update() {
        if (transform.position.y < yMin) {
            Destroy(this.gameObject);
        }
    }
}
```

运行，在 `Hierarchy` 面板中观察到苹果掉出屏幕外时被销毁。

3.3 篮筐的逻辑

3.3.1 生成篮筐

创建篮筐属于游戏主逻辑，需在游戏开始后生成竖直放置的三个篮筐。

`Logic_Main` 类的 `Start()` 函数中生成最小 `y` 值为 `basketMinY`、竖直间隔为 `basketSpacingY` 的 `numBaskets` 个篮筐，并自下而上加入私有成员变量 `basketList` 中，即 `basketList` 的最后一个元素是最上面的篮筐。

```
[Header(("Set in Inspector"))]
public GameObject basketPrefab; // 篮筐预设体
public int numBaskets = 3; // 篮筐数
public float basketMinY = -14.0f; // 篮筐最小 y
public float basketSpacingY = 3.0f; // 相邻篮筐的 y 间隔
```

```

List<GameObject> basketList;

void Start() {
    basketList = new List<GameObject>();
    for (int i = 0; i < numBaskets; i++) {
        GameObject tBasketGO = Instantiate(basketPrefab) as GameObject;
        Vector3 position = Vector3.zero;
        position.y = basketMinY + (basketSpacingY * i);
        tBasketGO.transform.position = position;
        basketList.Add(tBasketGO);
    }
}

```

将 Basket 的预设体拖到 Basket Prefab 上。如图 3.3.1.1 所示。

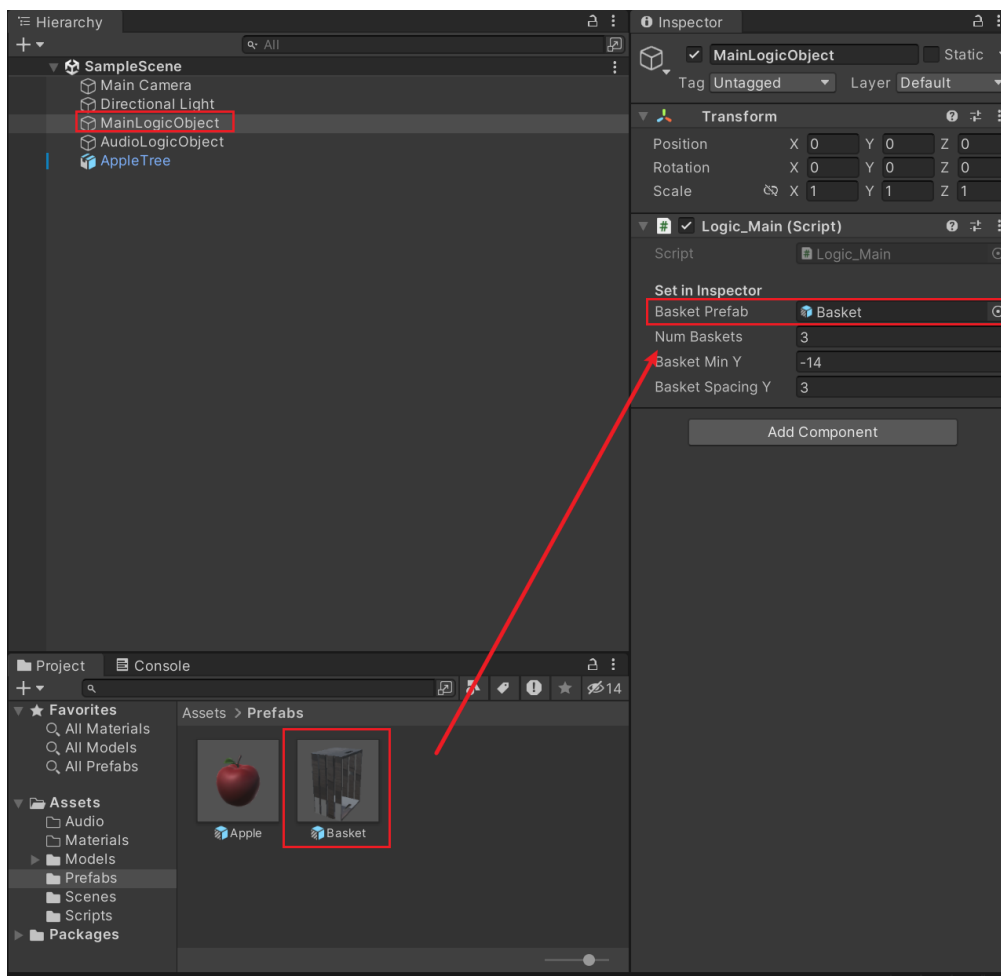


图 3.3.1.1: 将 Basket 的预设体拖到 Basket Prefab 上

运行,Game 面板中观察到画面中下方生成了 3 个可与苹果发生碰撞的篮筐。如图 3.3.1.2 所示。

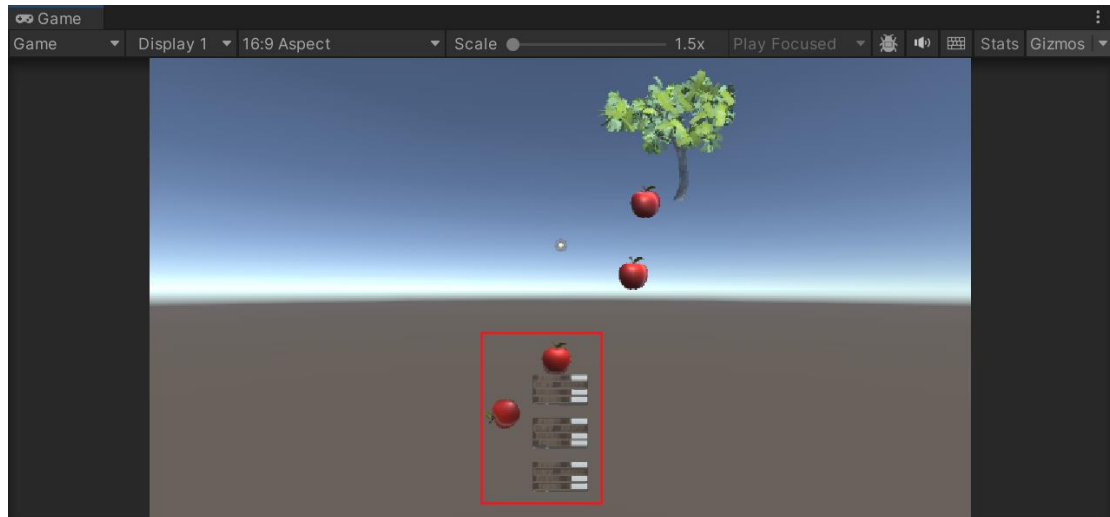


图 3.3.1.2: 生成了 3 个可与苹果发生碰撞的篮筐

3.3.2 篮筐跟随鼠标移动

新建脚本 `Logic_Basket.cs`，并挂在 `Basket` 的预设体上。

`Logic_Basket` 类的 `Update()` 函数获取鼠标在屏幕中的坐标（二 D）和摄像机的 `z` 坐标，将两坐标拼在一起后转化为三维的世界坐标，并修改篮筐的 `x` 坐标。

```
void Update() {
    Vector3 mousePosition2D = Input.mousePosition; // 鼠标的屏幕坐标

    // 摄像机的 z 坐标决定在三维空间中鼠标沿 z 轴向前移动多远
    mousePosition2D.z = -Camera.main.transform.position.z;

    // 将该点转化为三维的世界坐标
    Vector3 mousePosition3D = Camera.main.ScreenToWorldPoint(mousePosition2D);

    // 修改篮筐的 x 坐标
    Vector3 position = this.transform.position;
    position.x = mousePosition3D.x;
    this.transform.position = position;
}
```

运行，在 `Game` 面板中观察到 3 个篮筐跟随鼠标移动。如图 3.3.2.1 所示。

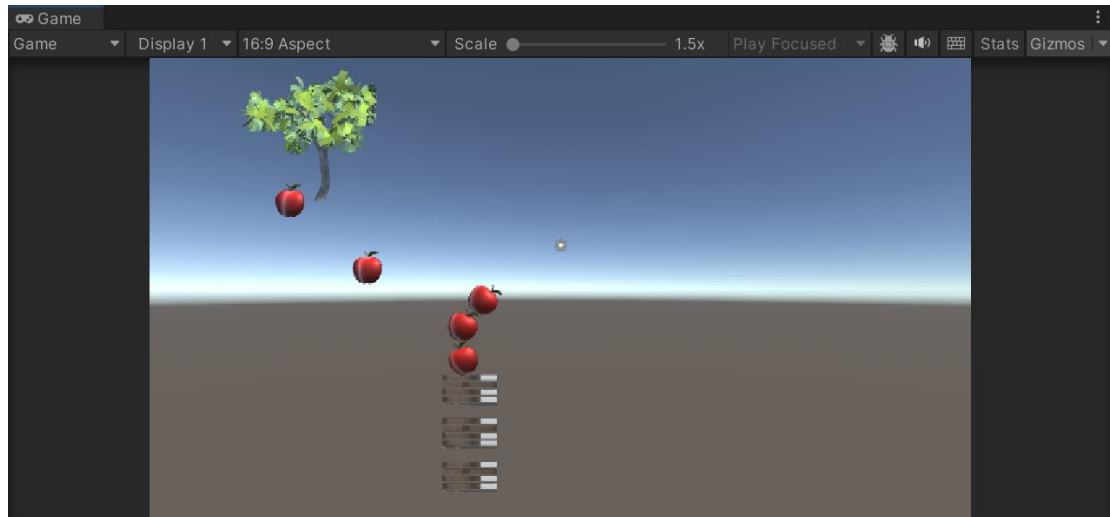


图 3.3.2.1: 3 个篮筐跟随鼠标移动

3.3.3 碰撞时销毁苹果

在 `Logic_Basket` 类中加入成员函数 `OnCollisionEnter()`，判断碰撞对象，若是苹果，则销毁碰撞对象。

```
void OnCollisionEnter(Collision collision) {  
    // 若碰撞的是苹果, 则销毁苹果  
    GameObject collidedWith = collision.gameObject;  
    if (collidedWith.tag == "Apple") {  
        Destroy(collidedWith);  
    }  
}
```

运行，在 `Game` 面板中观察到与篮筐碰撞的苹果被销毁。

4. 计分板

4.1 创建计分板

在 Hierarchy 面板的空白处右键，点击 UI/Text-TextMeshPro，添加一个 Text 组件。如图 4.1.1 所示。

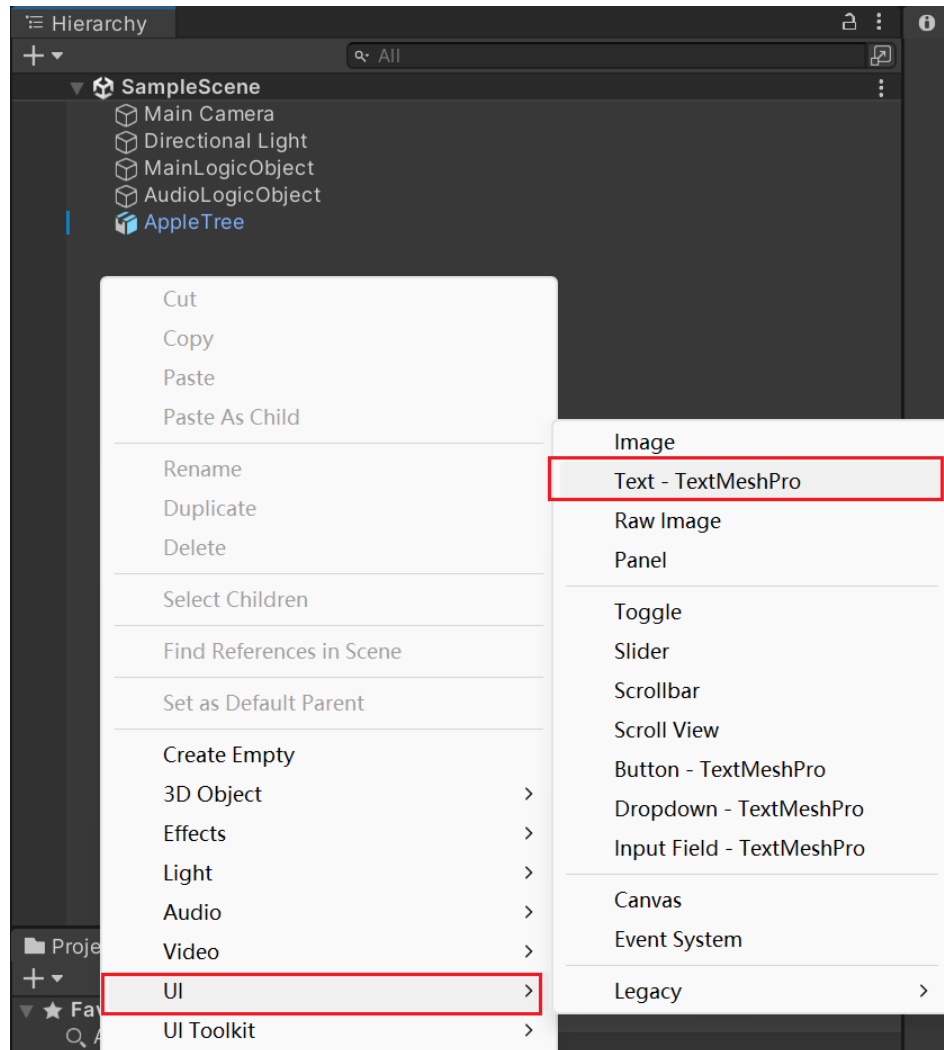


图 4.1.1: 添加 Text 组件

Unity 会自动生成一个空物体 Canvas，并添加的 Text 组件作为其儿子。将其重命名为“HighScore”。如图 4.1.2 所示。



图 4.1.2: 将 Text 组件重命名为“HighScore”

调整 HighScore 的位置、文本、字体、大小等属性，让它显示在画面的左上角。如图 4.1.3 所示。

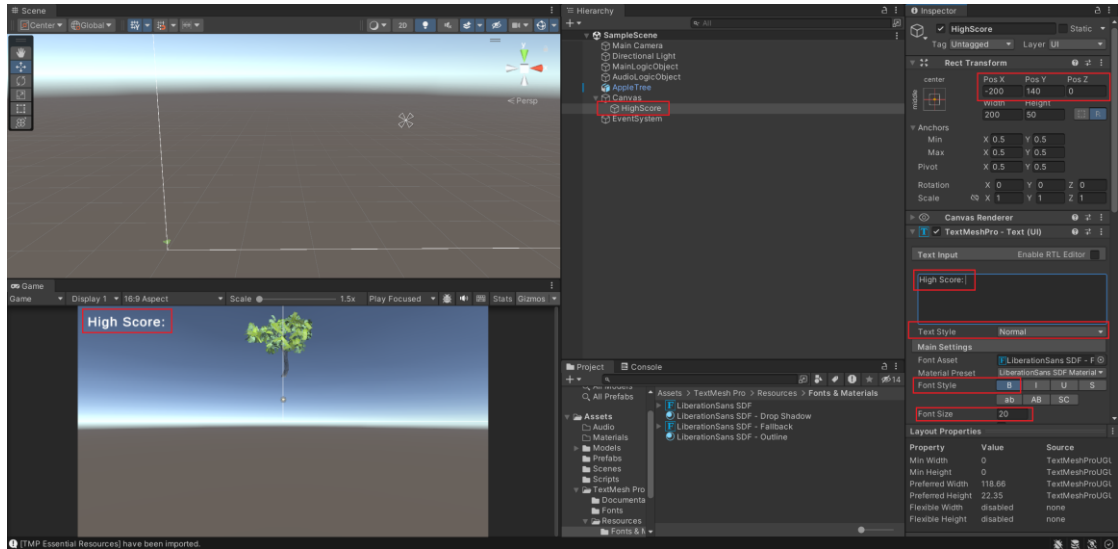


图 4.1.3: 让 HighScore 显示在画面的左上角

在 Hierarchy 面板中选择 HighScore, Ctrl + D 复制一份, 重命名为 CurrentScore。调整其位置、文本等, 让它显示在画面的右上角。如图 4.1.4 所示。

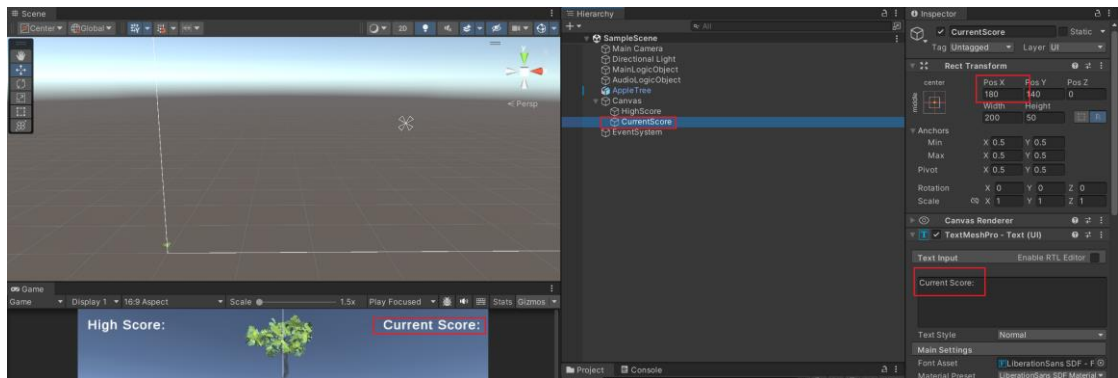


图 4.1.4: 让 CurrentScore 显示在画面的左上角

后续得分的数字的位数可能增加, 为使得 CurrentScore 的位置可自适应, 设置其 Alignment 为右对齐即可。如图 4.1.5 所示。

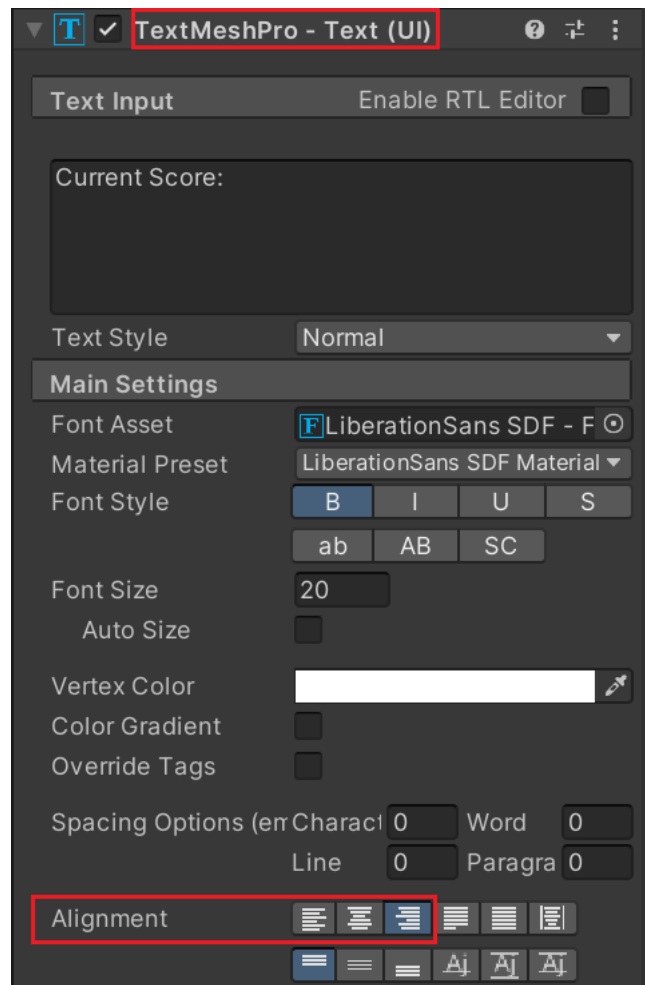


图 4.1.5: 设置 CurrentScore 为右对齐

4.2 CurrentScore 的初始化与更新

在 Hierarchy 面板中新建空物体 ScoreLogicObject，新建脚本 Logic_Score.cs 并挂在 ScoreLogicObject 上。

Logic_Score 类中：

- (1) 静态成员变量 currentScore 用于记录当前得分。
- (2) 公有静态成员函数 GetCurrentScoreText() 获取场景中的 CurrentScore 对象。
- (3) 公有静态成员函数 UpdateCurrentScore() 更新 currentScore 和 CurrentScore。
- (4) Start() 函数中初始化 currentScore。

```
static int currentScore; // 当前得分

public static int GetCurrentScore() {
    return currentScore;
}

static TMP_Text GetCurrentScoreText() {
    return GameObject.Find("CurrentScore").GetComponent<TMP_Text>();
}
```

```

public static void UpdateCurrentScore(int _currentScore) {
    currentScore = _currentScore;
    TMP_Text currentScoreText = GetCurrentScoreText();
    currentScoreText.text = "Current Score: " + currentScore.ToString();
}

void Start() {
    // 初始化 CurrentScore
    UpdateCurrentScore(0);
}

```

运行，在 Game 面板观察到 CurrentScore 对象的文本显示了当前得分，且 CurrentScore 的位置自适应。如图 4.2.1 所示。

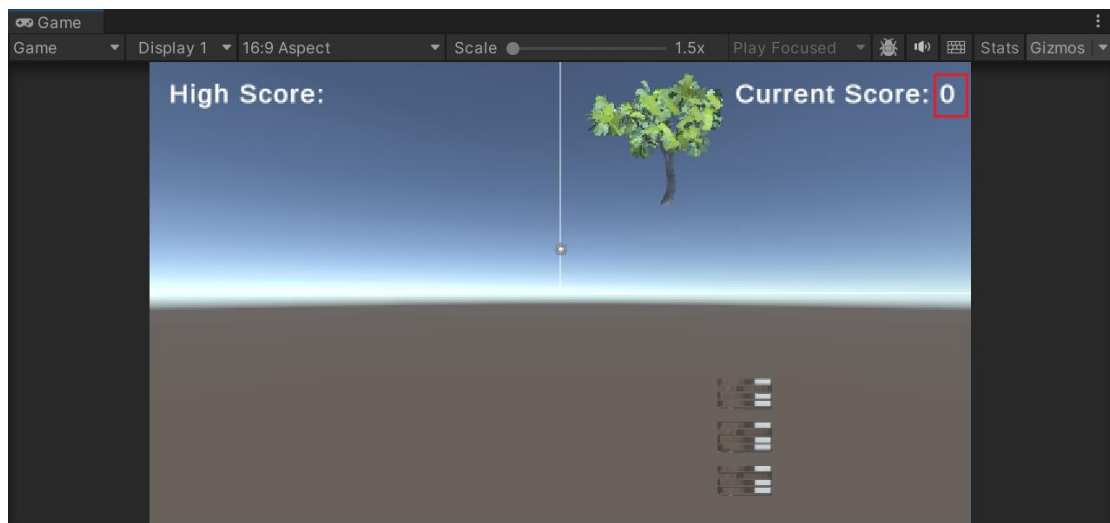


图 4.2.1: CurrentScore 的初始化

下面实现每接住 1 个苹果得 1 分。

在 Logic_Basket.cs 的 OnCollisionEnter()函数中加入。

```

void OnCollisionEnter(Collision collision) {
    ...
    if (collidedWith.tag == "Apple") {
        ...

        // 得 1 分
        Logic_Score.UpdateCurrentScore(Logic_Score.GetCurrentScore() + 1);
    }
}

```

运行，每接住 1 个苹果，得分加 1。如图 4.2.2 所示。

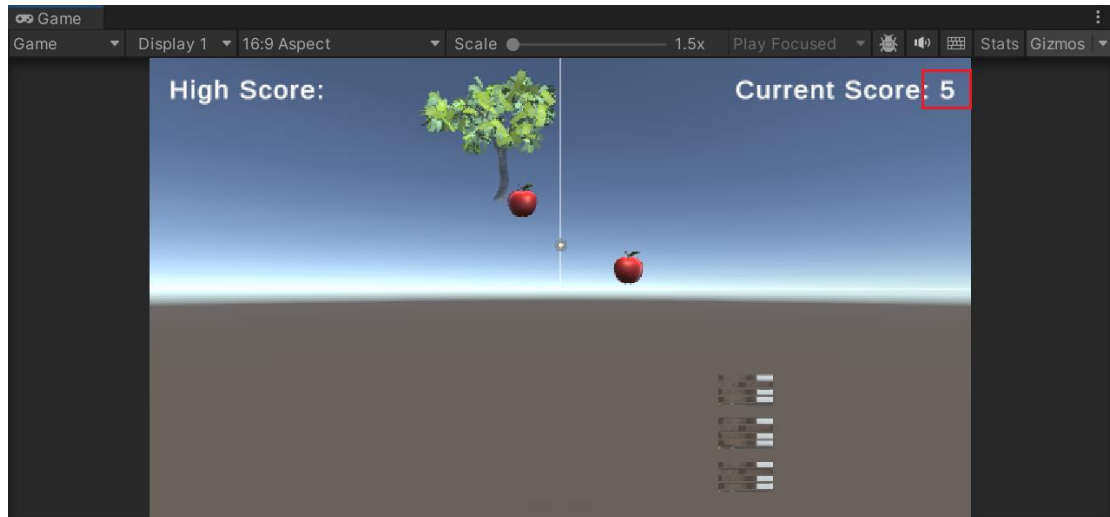


图 4.2.2: CurrentScore 的更新

4.3 HighScore 的初始化

《Apple Picker》希望 High Score 为玩家的历史最高分，且重新打开游戏时历史最高分不丢失。该功能可使用 PlayerPrefs 实现，它将信息保存在计算机的注册表中。

类似于 CurrentScore 的初始化，在 Logic_Score 类中加入。

```
...
static int highScore; // 历史最高得分

public static int GetHighScore() {
    return highScore;
}

static TMP_Text GetHighScoreText()
{
    return GameObject.Find("HighScore").GetComponent<TMP_Text>();
}

public static void UpdateHighScore(int _highScore) {
    highScore = _highScore;
    PlayerPrefs.SetInt("HighScore", highScore);

    TMP_Text highScoreText = GetHighScoreText();
    highScoreText.text = "High Score: " + highScore.ToString();
}

void Start() {
    ...

    // 初始化 HighScore
    highScore = 0;
    if (PlayerPrefs.HasKey("HighScore")) {
```

```
        highScore = PlayerPrefs.GetInt("HighScore");  
    }  
    UpdateHighScore(highScore);  
}
```

其中初始化 **HighScore** 时会先读取 **PlayerPrefs** 中是否有 **HighScore**，若有则读取并赋给 **HighScore**。

HighScore 计划在 **Game Over** 时更新，此处暂且跳过。

5. Game Over

5.1 漏接苹果的逻辑

5.1.1 概述

《Apple Picker》中，漏接 1 个苹果将丢失 1 个篮筐，即损失一条命。3 个篮筐用完后，Game Over，更新 HighScore 并重新开始游戏。

5.1.2 漏接苹果

若漏接 1 个苹果，在丢失 1 个篮筐的同时，还会清除所有正在下落的苹果，以降低游戏难度。

给 Logic_Main 类的成员变量 basketList 加上 static。

```
static List<GameObject> basketList;
```

在 Logic_Main 类中加入成员函数 DestroyAllApples()，其获取所有标签为 Apple 的游戏对象并销毁。

```
public static void DestroyAllApples() {  
    GameObject[] apples = GameObject.FindGameObjectsWithTag("Apple");  
    foreach (GameObject apple in apples) {  
        Destroy(apple);  
    }  
}
```

在 Logic_Main 类中加入成员函数 ConsumeABasket()，获取最上方的一个篮筐并销毁。

```
public static void ConsumeABasket() {  
    int topIndex = basketList.Count - 1;  
    GameObject topBasket = basketList[topIndex];  
    basketList.RemoveAt(topIndex);  
    Destroy(topBasket);  
}
```

在 Logic_Apple 类的 Update()函数中加入。

```
void Update() {  
    if (transform.position.y < yMin) {  
        ...  
  
        Logic_Main.DestroyAllApples();  
        Logic_Main.ConsumeABasket();  
    }  
}
```

运行，在 Game 面板中观察到漏接 1 个苹果会丢失 1 个篮筐，同时清除所有正在下落的苹果。如图 5.1.2.1 所示。

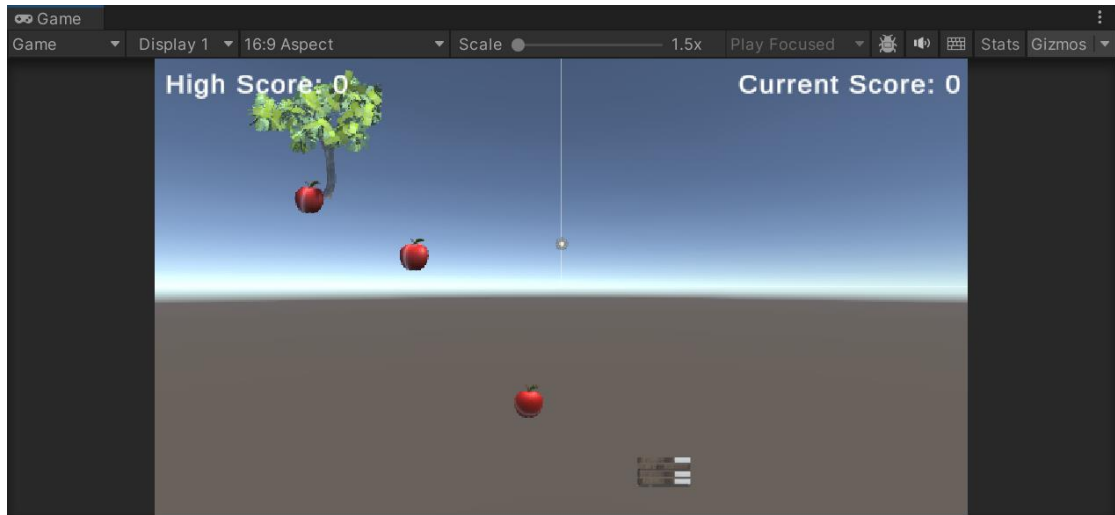


图 5.1.2.1: 漏接 1 个苹果会丢失 1 个篮筐, 同时清除所有正在下落的苹果

5.1.3 Game Over

3 个篮筐用完后, Game Over, 更新 HighScore 并重新开始游戏。

在 Logic_Main 类的 ConsumeABasket()函数中加入。

```
public static void ConsumeABasket() {
    ...

    // 若无篮筐, 则 Game Over
    if (topIndex == 0) {
        // 更新 HighScore
        int currentScore = Logic_Score.GetCurrentScore();
        if (currentScore > Logic_Score.GetHighScore()) {
            Logic_Score.UpdateHighScore(currentScore);
        }

        // 重新开始游戏
        SceneManager.LoadScene("SampleScene");
    }
}
```

运行, 在 Game 面板中观察到, 丢失所有篮筐后游戏重新开始, 同时更新 HighScore。如图 5.1.3.1 所示。

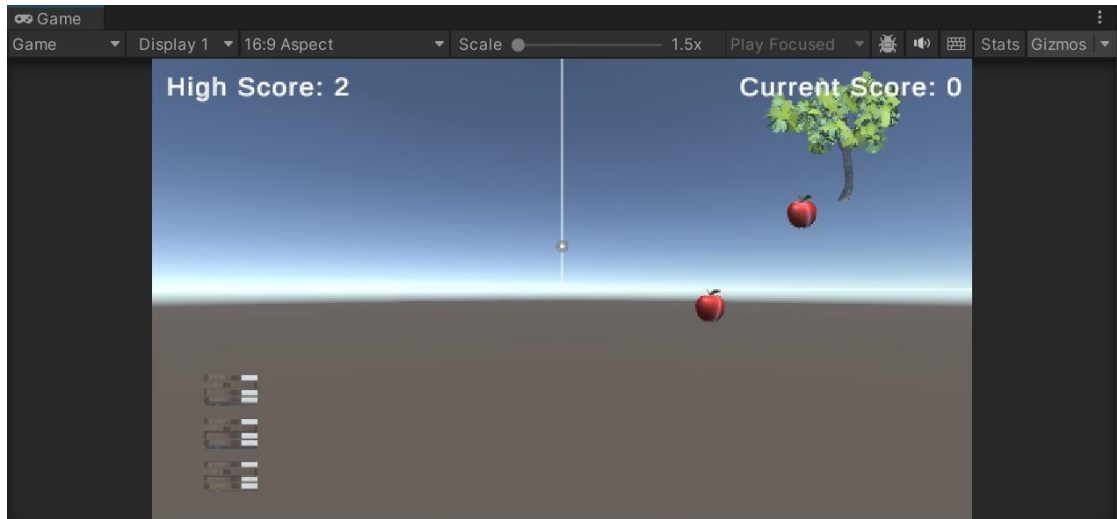


图 5.1.3.1: 丢失所有篮筐后游戏重新开始，同时更新 HighScore
关闭并重新运行游戏，发现 HighScore 仍保留。

5.2 结束界面

5.2.1 新建场景

将 Project 面板的 Assets/Scenes 的“SampleScene.unity”重命名为“GameScene.unity”，并新建场景“EndScene.unity”。如图 5.2.1.1 所示。

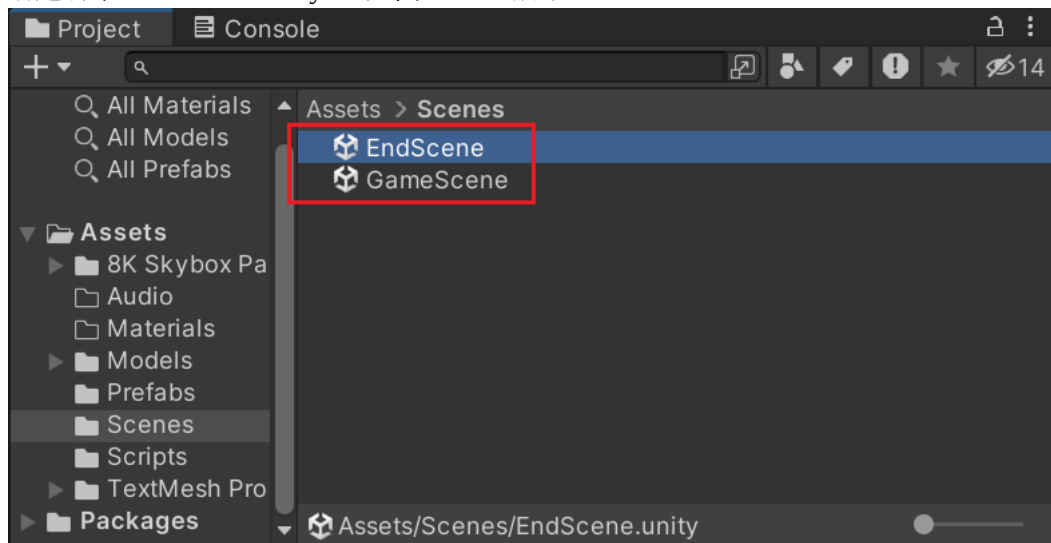


图 5.2.1.1: 重命名 SampleScene 并新建 EndScene

双击 EndScene.unity 进入场景。

5.2.2 添加背景

在 Hierarchy 面板的空白处右键，点击 UI/Raw Image，新建一个 Raw Image 对象，并重命名为“Background”。

在 Project 面板的 Assets/Pictures 文件夹中导入周深在 9.29Hz 巡回演唱会 – 深圳站演唱《借过一下》的饭拍截图。（图片来源：【周深】深圳演唱会开场唱庆余年 2《借过一下》好震撼！【2024 周深 9.29Hz 巡回演唱会】_4K 饭拍】
<https://www.bilibili.com/video/BV1P1421y7Nt>）

将上图拖到 Background 的 Raw Image/Texture 上，并调整 Background 的位置、宽高、大小。如图 5.2.2.1 所示。

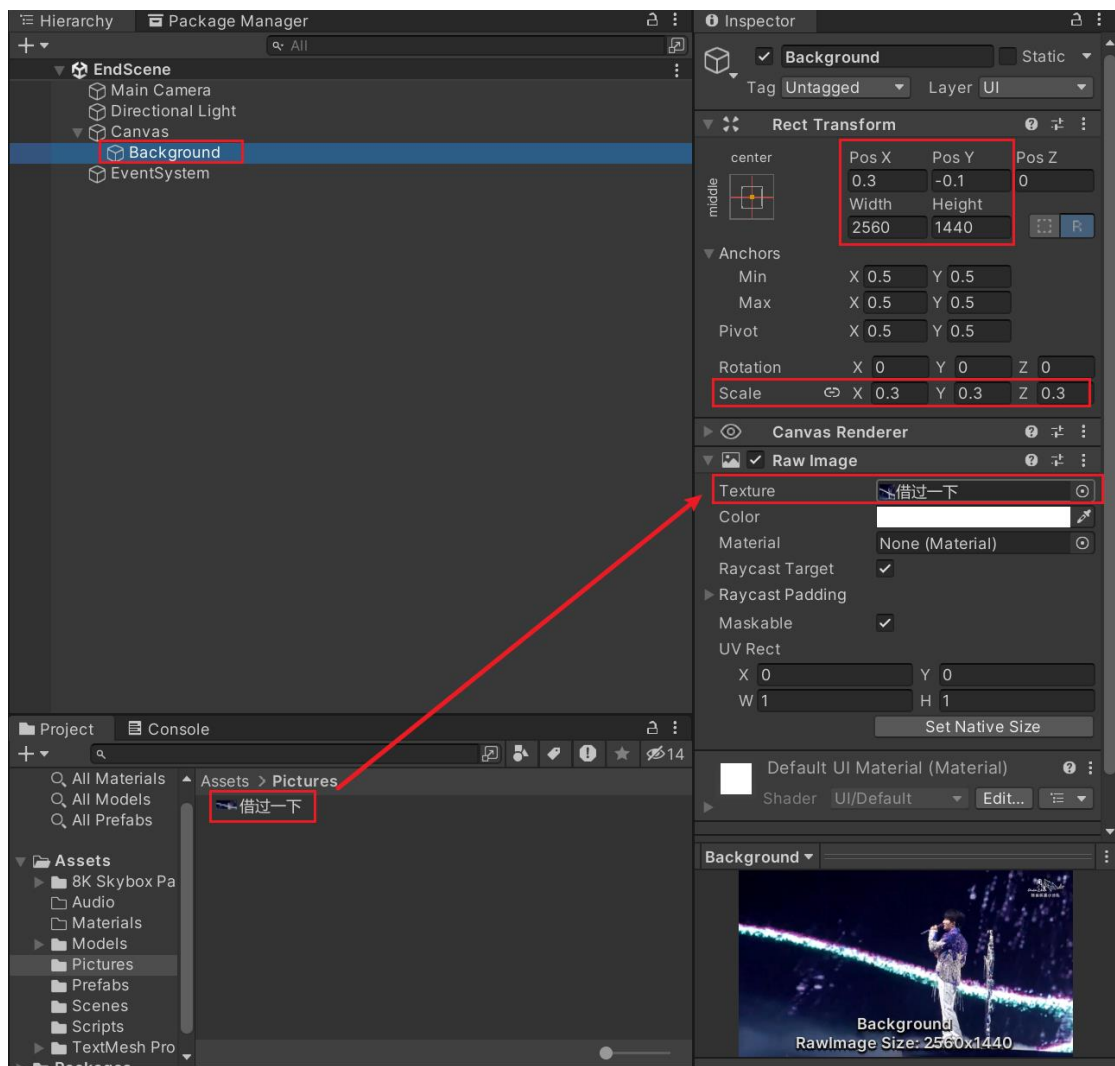


图 5.2.2.1: 调整 Background 的位置、宽高、大小

5.2.3 添加破纪录

类似地添加破纪录图片 BreakRecord。如图 5.2.3.1 所示。(图片来源:【花瓣网】<https://huaban.com/pins/798161098>)

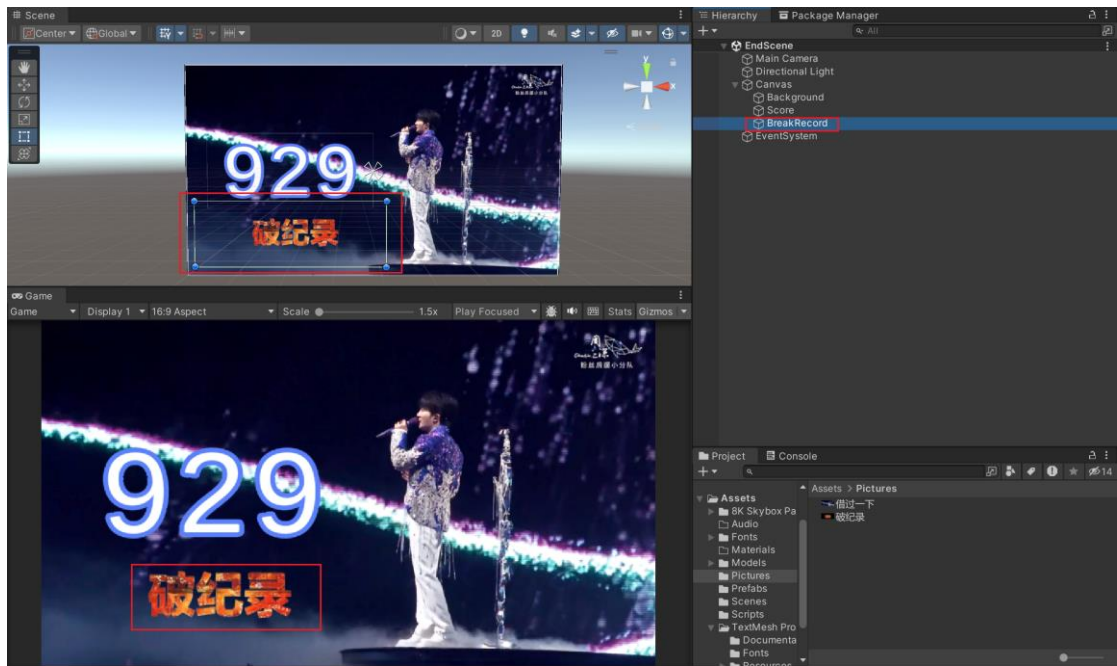


图 5.2.3.1: 添加破纪录图片 BreakRecord

5.2.3 添加重玩按钮

添加一个 Button – TextMeshPro 组件 Replay，设置按钮的背景色、按钮的文本和颜色。如图 5.2.3.1 所示。

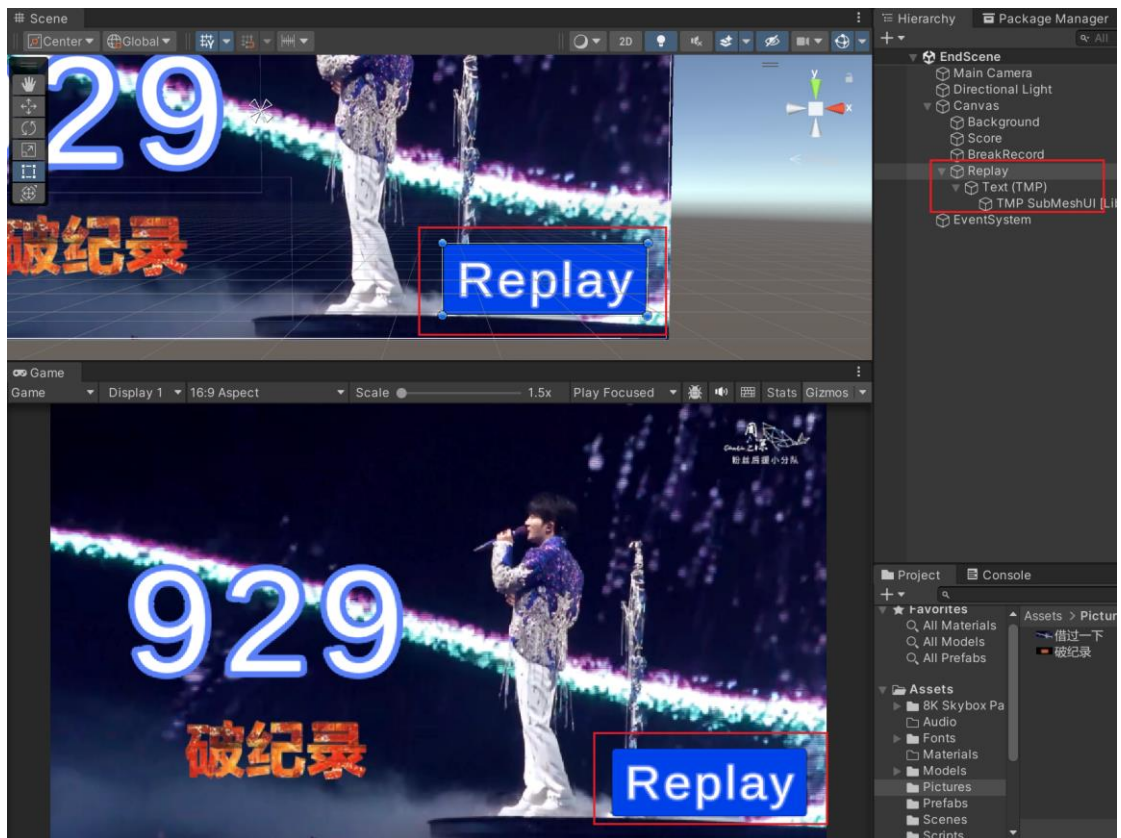


图 5.2.3.1: 设置 Replay 按钮的背景色、按钮的文本和颜色

新建脚本 Logic_ReplayButton，并挂在 Replay 上。

Logic_ReplayButton 类中添加公有函数 ReplayButtonClicked()，实现加载场景

GameScene。

```
public class Logic_ReplayButton : MonoBehaviour {  
    public void ReplayButtonClicked() {  
        // 重新开始游戏  
        SceneManager.LoadScene("GameScene");  
    }  
}
```

如图 5.2.3.2，在 Replay 按钮的 Button 组件中，点击红色框的加号，将蓝色框的脚本拖到绿色框中，在紫色框中选择 Logic_ReplayButton.ReplayButtonClicked()函数。

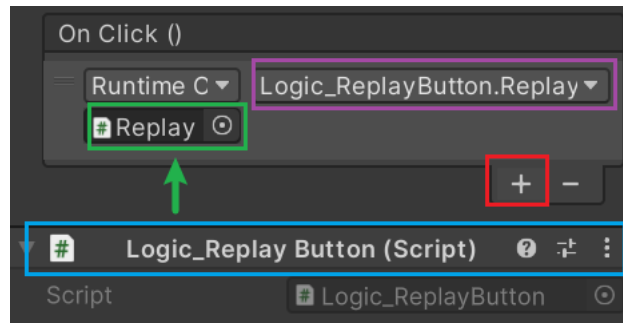


图 5.2.3.2: 设置 Replay 按钮点击事件

Ctrl + Shift + B 打开 Build Settings 窗口，将 Project 窗口的 Assets/Scene 文件夹中的 EndScene 和 GameScene 拖到 Scenes In Build 列表中，调整顺序使得 EndScene 为 0 号场景，即游戏运行时会先加载该场景。如图 5.2.3.3 所示。

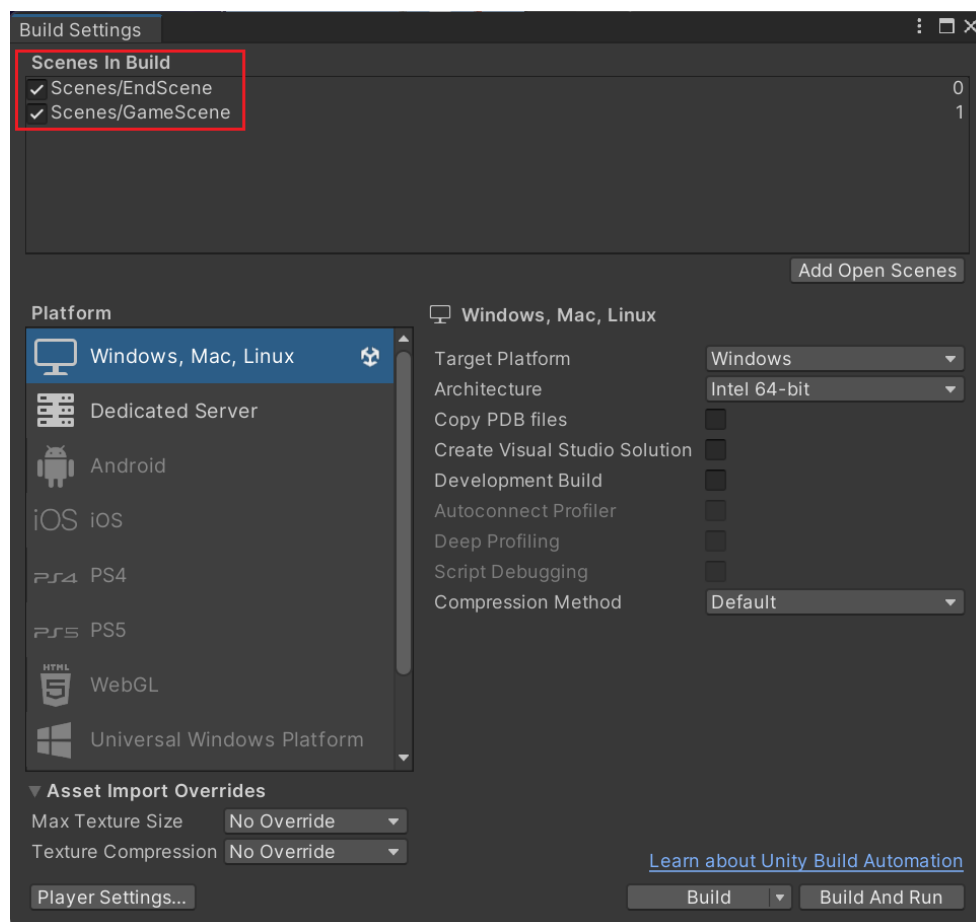


图 5.2.3.3: 添加 Scenes In Build

运行，点击 Replay 按钮即可跳转到 Game Scene，即重新开始游戏。

5.2.4 添加分数

添加一个 Text – TextMeshPro 组件 Score，调整其文本、大小、粗细、对齐方式等。如图 5.2.4.1 所示。

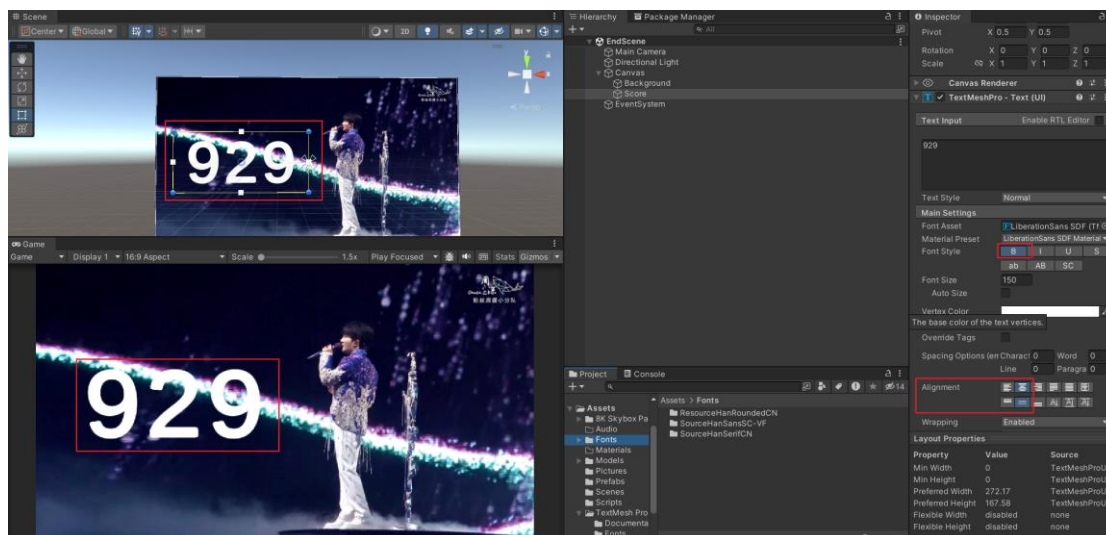


图 5.2.4.1: 调整 Score 的文本、大小、粗细、对齐方式等

在 LiberationSans SDF Material 中开启描边，设置表面的颜色和粗细。如图 5.2.3.2 所示。

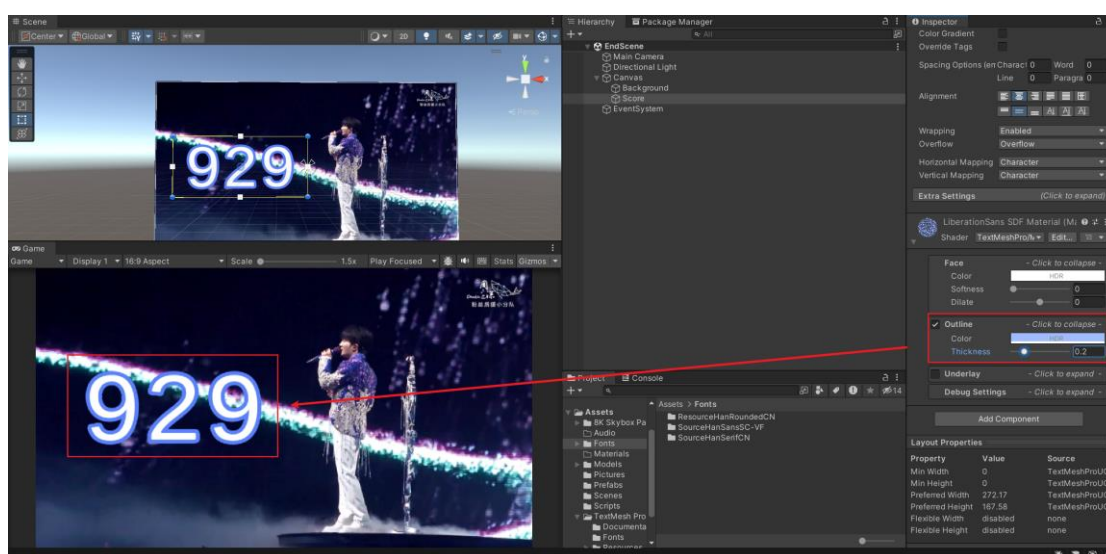


图 5.2.4.2: 添加描边

在 Hierarchy 面板中新建空对象 MainLogicObject，新建脚本 Logic_EndScene.cs，并挂在 MainLogicObject 上。

在 Logic_EndScene 类的 Start()函数中，获取并显示当前分数、隐藏破纪录图片，并根据是否破记录决定是否显示破纪录图片和更新 HighScore。

```
static TMP_Text GetScoreText() {
    return GameObject.Find("Score").GetComponent<TMP_Text>();
}

void Start() {
```

```

// 显示当前分数
int currentScore = Logic_Score.GetCurrentScore();
TMP_Text score = GetScoreText();
score.text = currentScore.ToString();

// 隐藏破纪录图片
GameObject breakRecord = GameObject.Find("BreakRecord");
breakRecord.SetActive(false);

// 若破纪录, 显示破纪录的图片, 并更新 HighScore
if (currentScore > Logic_Score.GetHighScore()) {
    breakRecord.SetActive(true);
    Logic_Score.UpdateHighScore(currentScore);
}
}

```

修改 Logic_Main 类的 ConsumeABasket()函数, 当 topIndex == 0 时, 先不更新 HighScore, 只跳转到 EndScene。

```

public static void ConsumeABasket() {
    ...

    // 若无篮筐, 则 Game Over
    if (topIndex == 0) {
        /*// 更新 HighScore
        int currentScore = Logic_Score.GetCurrentScore();
        if (currentScore > Logic_Score.GetHighScore()) {
            Logic_Score.UpdateHighScore(currentScore);
        }

        // 重新开始游戏
        SceneManager.LoadScene("GameScene");*/

        // 切换到 EndScene
        SceneManager.LoadScene("EndScene");
    }
}
}

```

运行：

(1) 游戏得 1 分时, Game Over 后跳转到 EndScene, 不显示打破纪录的图片。如图 5.2.4.3 和图 5.2.4.4 所示。

(2) 点击 Replay 按钮重新开始游戏。

(3) 游戏得 7 分时, Game Over 后跳转到 EndScene, 显示打破纪录的图片。如图 5.2.4.5 和图 5.2.4.6 所示。



图 5.2.4.3: 游戏得 1 分

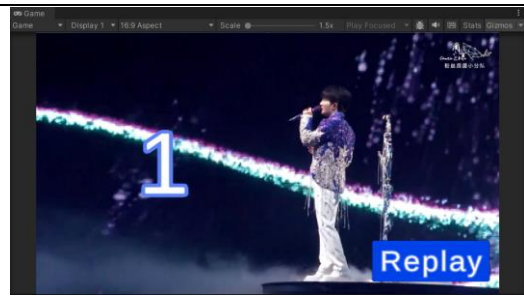


图 5.2.4.4: 未打破纪录



图 5.2.4.5: 游戏得 7 分

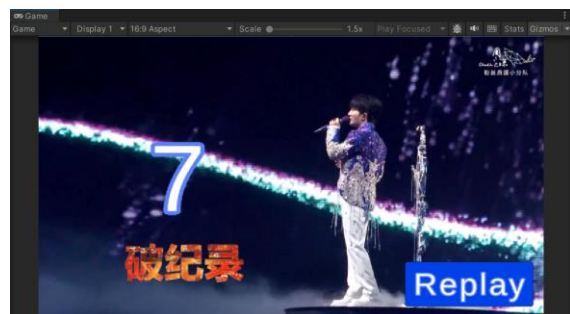


图 5.2.4.6: 打破纪录

在 Logic_EndScene 类中调用 UpdateHighScore()无法获得 GameScene 中的 HighScore 对象，故会产生空引用的错误。为此，修改 Logic_Score 类的 UpdateHighScore()函数。

```
public static void UpdateHighScore(int _highScore) {
    ...

    TMP_Text highScoreText = GetHighScoreText();
    if (highScoreText != null) {
        highScoreText.text = "High Score: " + highScore.ToString();
    }
}
```


6. 开始界面

6.1 新建场景

在 Project 面板的 Assets/Scenes 文件夹中新建场景 StartScene.unity，并双击打开。

类似于 5.2 结束界面，为 StartScene 加入一个 RawImage 组件，重命名为“Background”，并调整大小为 820 x 440，为其添加合适的图片。如图 6.1.1 所示。



图 6.1.1：开始界面背景

上述图片的背景采用 Stable Diffusion 绘制，其 Prompt 如下：

(masterpiece:1,2),best quality,highres,original,extremely detailed wallpaper,perfect lighting,(extremely detailed CG:1.2),drawing,paintbrush,2girls,loli,((red apple)),holding_fruit,
Negative prompt: (NSFW:1.5),(worst quality:2),(low quality:2),(normal quality:2),lowres,normal quality,((monochrome)),((grayscale)),skin spots,acnes,skin blemishes,age spot,(ugly:1.331),(duplicate:1.331),(morbid:1.21),(mutilated:1.21),(tranny:1.331),mutated hands,(poorly drawn hands:1.5),blurry,(bad anatomy:1.21),(bad proportions:1.331),extra limbs,(disfigured:1.331),(missing arms:1.331),(extra legs:1.331),(fused fingers:1.61051),(too many fingers:1.61051),(unclear eyes:1.331),lowers,bad hands,missing fingers,extra digit,bad hands,missing fingers,(((extra arms and legs))),
Steps: 20, Sampler: DPM++ 2M Karras, CFG scale: 7, Seed: 1604052237, Size: 768x512, Model hash: a074b8864e, Model: Anime - Counterfeit V2.5, Clip skip: 2, Version: v1.7.0

同理再添加一个 RawImage 组件，重命名为“StartButton”，为其添加合适的图片并移动到合适的位置。

在 Hierarchy 面板中选中“StartButton”，在 Inspector 面板中点击 Add Component，点击 UI/Button，为其添加 Button 组件。

6.2 开始按钮点击事件

在 Project 面板的 Assets/Scripts 文件夹中选中“Logic_ReplayButton”，Ctrl + D 复制一份

并重命名为“Logic_StartButton”。稍加修改 Logic_StartButton.cs。

```
public class Logic_StartButton : MonoBehaviour {  
    public void StartButtonClicked() {  
        // 开始游戏  
        SceneManager.LoadScene("GameScene");  
    }  
}
```

类似于 5.2.3 添加重玩按钮，为 StartButton 绑定点击事件，如图 6.2.1 所示。

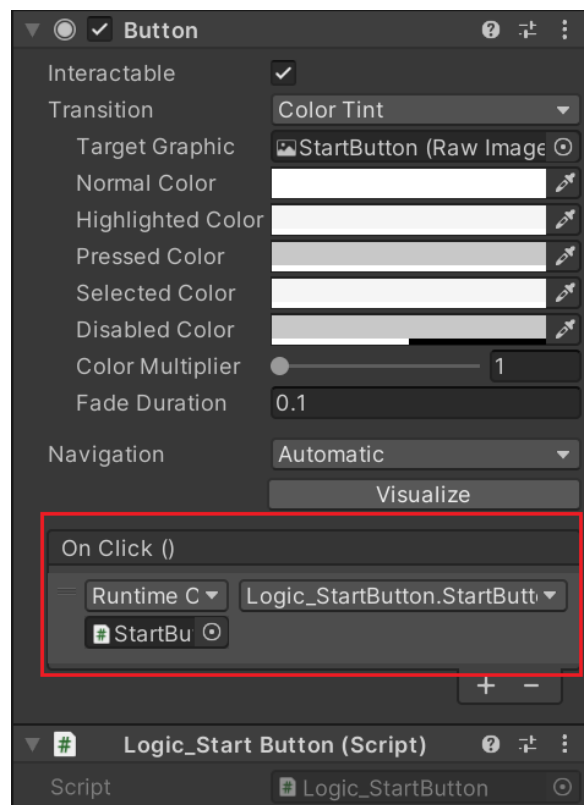


图 6.2.1：为 StartButton 绑定点击事件

类似于 5.2.3 添加重玩按钮，将 StartScene 加入 Build Settings 中。如图 6.2.2 所示。

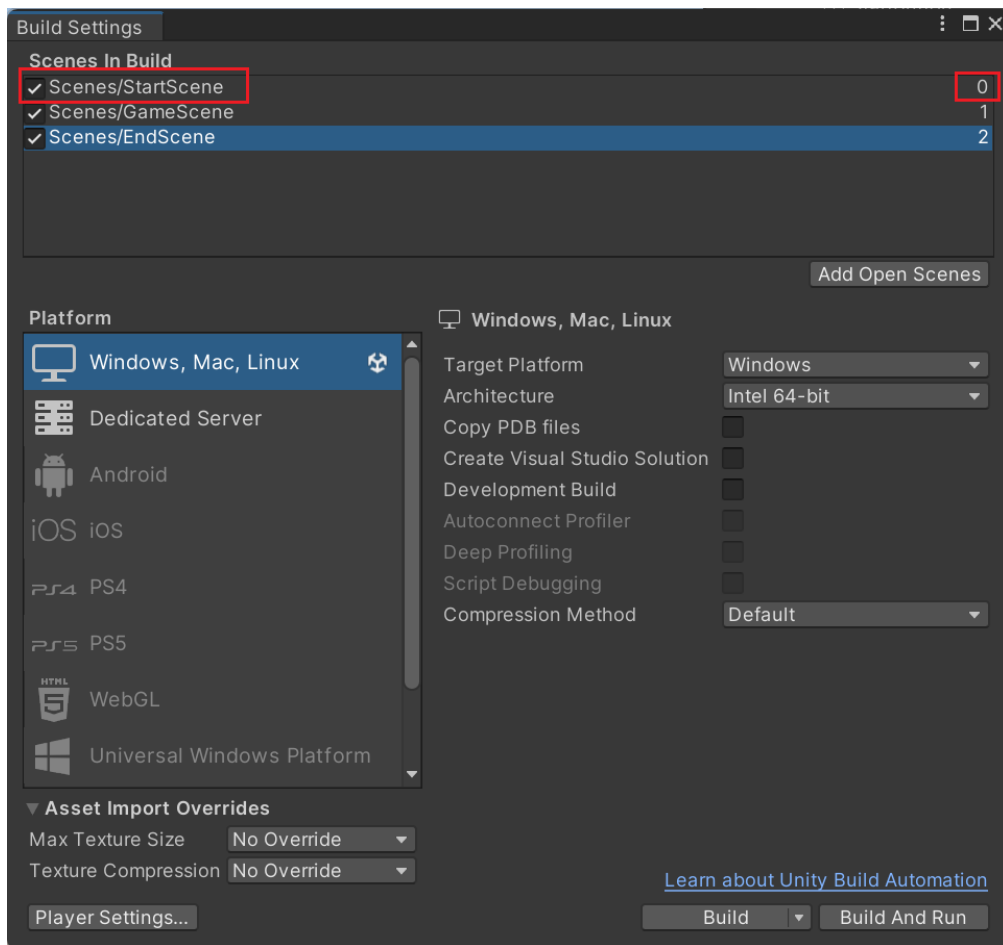


图 6.2.2: 将 StartScene 加入 Build Settings 中运行，点击 StartButton 即跳转到 GameScene。

7. 修复游戏 Bug

篮筐跟随鼠标移动时可能会移动到画面外。如图 7.1 所示。

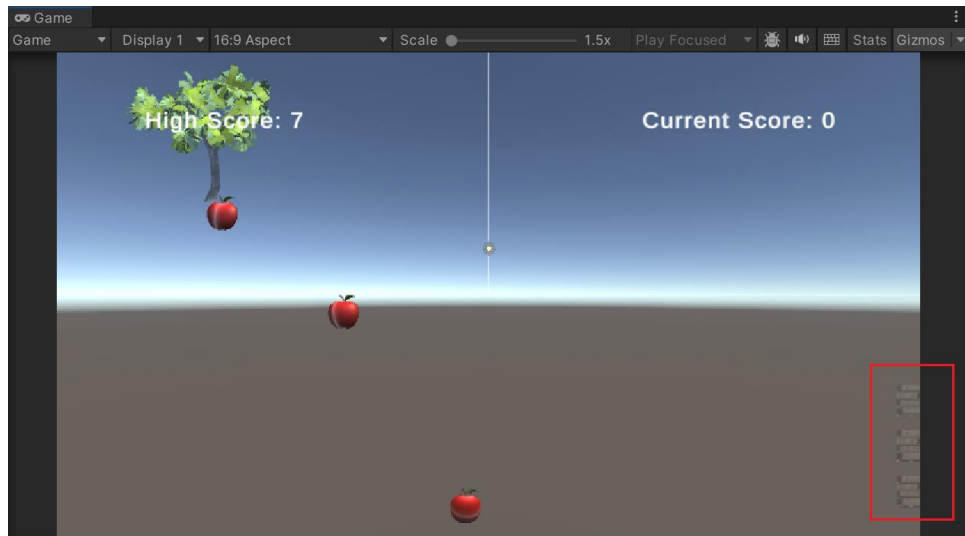


图 7.1: 篮筐跟随鼠标移动时可能会移动到画面外
在 Logic_Basket 类中加入 x 坐标的限制。

```
[Header("Set in Inspector")]
public float xRange = 20.0f;

void Update() {
    ...

    position.x = mousePosition3D.x;
    position.x = Mathf.Max(position.x, -xRange);
    position.x = Mathf.Min(position.x, xRange);
    this.transform.position = position;
}
```

运行，此时篮筐最右只能移动到图 7.2 所示的位置。

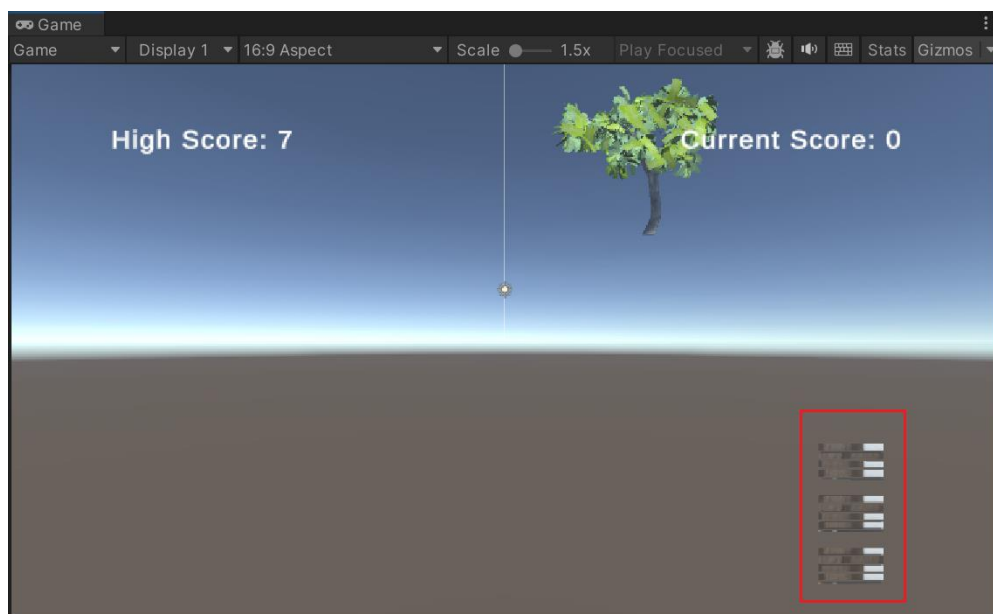


图 7.2: 篮筐最右移动位置

8. 游戏优化

8.1 天空盒

在 Unity 的 AssetStore 下载一套免费的 8K 天空盒 8K Skybox Pack Free（链接：<https://assetstore.unity.com/packages/2d/textures-materials/sky/8k-skybox-pack-free-150926>）。

Ctrl + 9 打开 Lightning 面板, 将天空盒的材质拖到 Skybox Material 上, 即可加载天空盒。如图 8.1.1 所示。

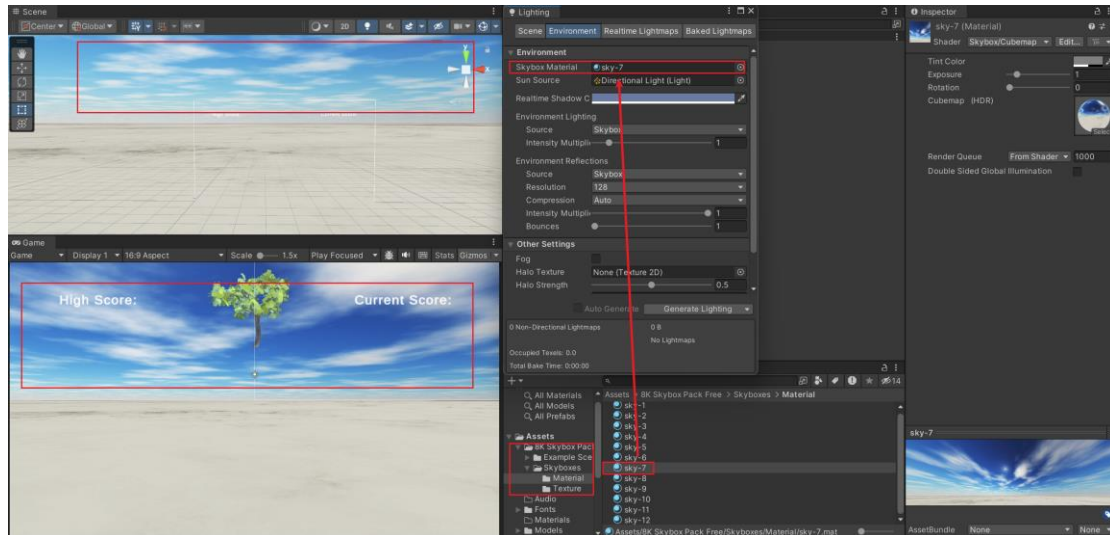


图 8.1.1: 加载天空盒

8.2 BGM 与音效

8.2.1 BGM

为 GameScene 添加 BGM 《向光而行》- 周深, 因为我写这个实验的时候在听这首歌。（音源：【【周深】《向光而行》MV（《这十年·追光者》节目主题曲）】<https://www.bilibili.com/video/BV1y14y1475C>）

为 AudioLogicObject 对象添加子对象 BGMLogicObject, 并为子对象添加 Audio/Source 组件。如图 8.2.1.1 所示。

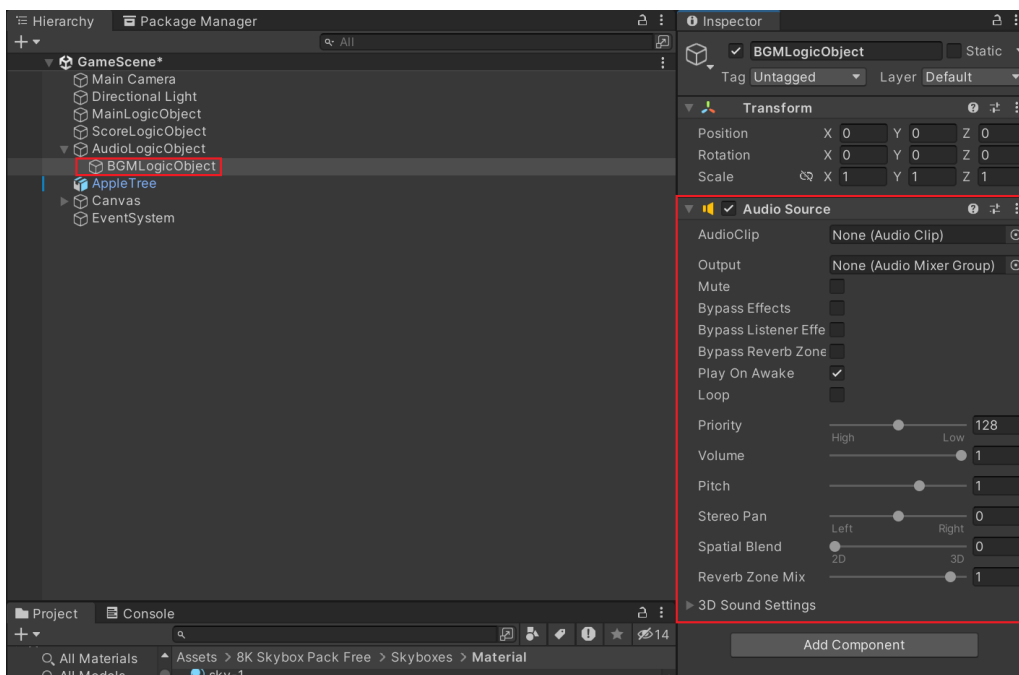


图 8.2.1.1: 为 BGMLogicObject 对象添加 Audio/Source 组件

将 BGM 的音源拖到 Audio Source.AudioClip 上，并启用 Play On Awake（场景 Awake 时播放）和 Loop（音乐循环播放）。如图 8.2.1.2 所示。

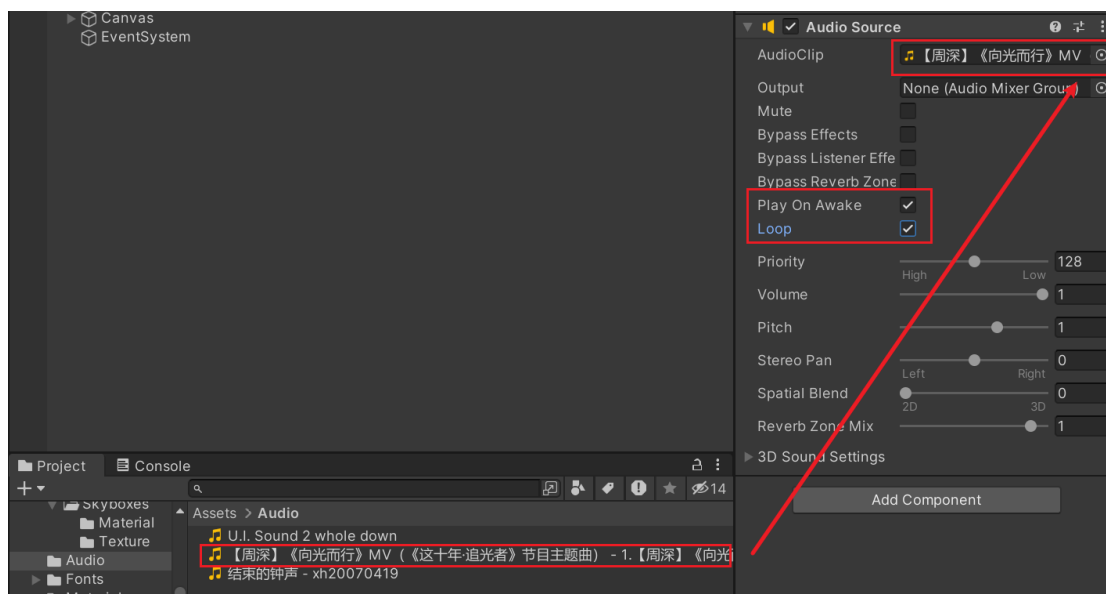


图 8.2.1.2: 设置 BGMLogicObject 对象的 Audio Source 组件

运行，加载 GameScene 时即播放 BGM。

8.2.2 音效

下面实现接到苹果时播放音效“噔”的一声。

类似地，为 AudioLogicObject 对象添加子对象 SoundEffectLogicObject，并为子对象添加 Audio/Source 组件。将音源拖到 Audio Source.AudioClip 上，关闭 Play On Awake，设置 Volume 为 0.5。如图 8.2.2.1 所示。

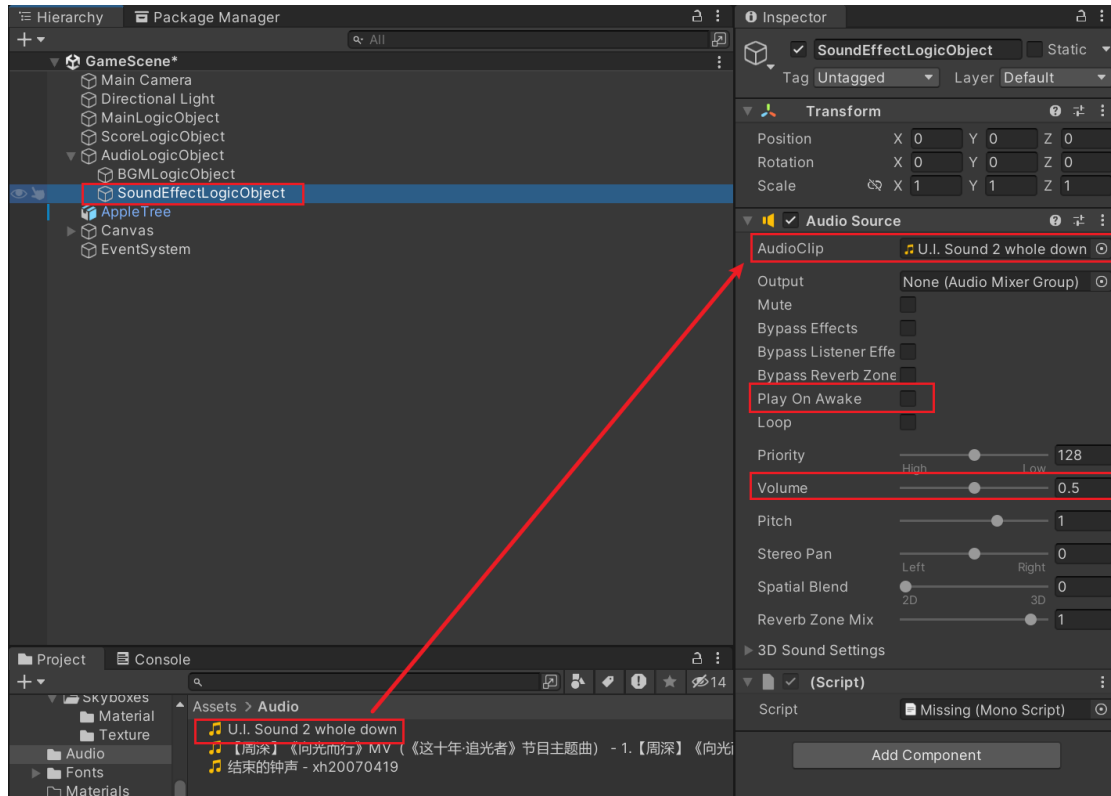


图 8.2.2.1: 添加并设置 SoundEffectLogicObject 对象
在 Logic_Basket 类中加入。

```
[Header("Set in Inspector")]
...
GameObject soundEffectObject;

void Start() {
    soundEffectObject = GameObject.Find("SoundEffectLogicObject");
}

void OnCollisionEnter(Collision collision) {
    if (collidedWith.tag == "Apple") {
        ...

        // 播放音效
        soundEffectObject.GetComponent().Play();
    }
}
```

运行，篮筐接住苹果后即播放音效。

8.3 键盘交互

下面实现在 GameScene 按 ESC 键暂停或继续游戏，同时暂停或继续播放 BGM。
在 Logic_Main 类中加入。

```
GameObject bgmObject;
```



```

void Start() {
    ...

    bgmObject = GameObject.Find("BGMLogicObject");
}

void Update() {
    if (Input.GetKeyDown(KeyCode.Escape)) {
        if (Time.timeScale > 0.01) {
            Time.timeScale = 0.0f;
            bgmObject.GetComponent<AudioSource>().Pause();
        }
        else {
            Time.timeScale = 1.0f;
            bgmObject.GetComponent<AudioSource>().UnPause();
        }
    }
}
}

```

其中 `Time.timeScale` 描述时间流速，默认为 `1.0f`。将其设为 `0.0f` 时游戏暂停。

8.4 颠倒重力方向

下面实现每隔 10 s 会有 1.5 s 的重力方向颠倒时间，这样牛顿的棺材板就压不住了。如图 8.4.1 所示。



图 8.4.1：牛顿的棺材板压不住了

在 `Logic_Main` 类中加入。

```
[Header(("Set in Inspector"))]
```

```

...
public float gravityDownTime = 10.0f; // 重力向下的持续时间
public float gravityUpTime = 1.5f; // 重力向上的持续时间

void Start() {
    ...

    Invoke("ReverseGravityDirection", gravityDownTime);
}

void ReverseGravityDirection() {
    Vector3 gravity = Vector3.zero;
    gravity.y = -Physics.gravity.y;
    if (Physics.gravity.y < 0) { // 当前向下
        Invoke("ReverseGravityDirection", gravityUpTime);
    }
    else { // 当前向上
        Invoke("ReverseGravityDirection", gravityDownTime);
    }
    Physics.gravity = gravity;
}
}

```

运行，发现每隔 10 s 会出现 1.5 s 的重力往上时间，原本正在作向下的匀加速直线运动的苹果变为作向上的匀加速直线运动。

8.5 难度递增

“引入不同种类的苹果，分别对应不同的得分；引入炸弹，碰到即 Game Over 或清空分数；……”这些提升难度的优化太过于 Naive 了，所以本次实验中不采用这些优化。

下面实现每接住一个苹果，苹果的掉落间隔会缩短 0.01 s，最小缩短至 0.2 s。

在 Logic_Basket 类的 OnCollisionEnter() 函数中加入。

```

void OnCollisionEnter(Collision collision) {
    ...
    if (collidedWith.tag == "Apple") {
        ...

        // 苹果掉落加速
        float timeInterval2DropApples =
appleTreeObject.GetComponent<Logic_AppleTree>().timeInterval2DropApples;
        appleTreeObject.GetComponent<Logic_AppleTree>().timeInterval2DropApples =
Mathf.Max(timeInterval2DropApples - 0.01f, 0.2f);
    }
}

```

修改 Logic_AppleTree 的 Start() 函数和 DropApple() 函数。

```

void Start() {
    // InvokeRepeating("DropApple", delay2DropFirstApple, timeInterval2DropApples);
}

```

```
    Invoke("DropApple", delay2DropFirstApple);  
}  
  
void DropApple() {  
    ...  
  
    Invoke("DropApple", timeInterval2DropApples);  
}
```

运行，发现苹果掉落越来越快，可在 AppleTree 的 Inspector 窗口观察到 timeInterval2DropApples 逐渐减小，最后时间间隔为 0.2 s，要全接住已非常困难。

四、实验结论或心得体会

9. 实验结论

运行录屏见【附件】Apple Picker.mp4。

- (1) Unity 是一款跨平台的游戏开发引擎，由 Unity Technologies 开发和维护。它提供了一个集成开发环境 (IDE)，支持 2D 和 3D 游戏的制作，并且可以发布到多个平台，包括 PC、移动设备、控制台和网页。Unity 拥有强大的图形渲染能力、物理引擎和丰富的插件资源，使得开发者能够高效地创建各种类型的游戏。
- (2) 使用 Unity 开发《Apple Picker》游戏相较于使用 Cocos2d-x 开发的优点：
 - ① Unity 提供了更加直观的开发环境和工具，使得开发效率更高。Cocos2d-x 虽然也很强大，但它更偏向于底层开发，需要更多的编程基础和时间投入。
 - ② Unity 的学习曲线较为平缓，适合初学者。而 Cocos2d-x 使用 C++ 编程，虽然灵活性高，但学习成本较高，对初学者不太友好。
 - ③ Unity 拥有更大的社区和更丰富的第三方资源，遇到问题时更容易找到解决方案。Cocos2d-x 的社区相对较小，资源也相对有限。
 - ④ Unity 提供了强大的可视化编辑器，开发者可以通过拖放的方式进行开发。而 Cocos2d-x 更多依赖代码实现，可视化程度较低。
 - ⑤ Unity 的 Asset Store 中有大量的插件和扩展，可以大大简化开发过程。Cocos2d-x 的扩展性虽然也很强，但插件数量和质量相对较少。

10. 实验心得

- (1) 用 Unity 做游戏可比用 Cocos2d-x 做爽多了。
- (2) 什么年代了还在用 Cocos2d-x，早该用 Unity 做游戏了。
- (3) 但用 Cocos2d-x 做期末大作业也不是一无是处，可以复习一遍 C++ 的多文件编程、面向对象编程的部分。对存档和读档功能，还加深了对外部变量、静态变量、公有变量等的理解。
- (4) 周深的歌真好听。

指导教师批阅意见： 成绩评定： 评语： 指导教师签字： 年 月 日
备注：

评语:

指导教师签字:

年 月 日

备注:

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。